Ch 7
Software Security

Reference:

https://spring.academy/courses

What Is Security?

- Software Security can mean many things. In this lesson, we'll talk about Web App Security.
- More specifically we'll cover:
  - how HTTP Authentication and Authorization work,
  - common ways in which the web ecosystem is vulnerable to attacks, and how we can use Spring Security to prevent unauthorized access to our Web REST API service.

Authentication

- A user of an API can actually be a person or another program, so, often we'll use the term '**Principal'** as a synonym for 'user'.
- **Authentication** is the act of a ***Principal*** proving its identity to the system. One way to do this is to provide credentials (e.g. a username and password using **Basic Authentication**).
- We say that once the proper credentials have been presented, the Principal is authenticated, or in other words, the user has successfully logged in.

Authentication (2)

- HTTP is a **stateless** protocol, so each request must contain data that proves it's from an authenticated Principal.
- Although it's possible to present the credentials on every request, doing so is inefficient because it requires more processing on the server. Instead, an **Authentication Session** (or Auth Session, or just Session) is created when a user gets authenticated.
- To implement sessions we'll use a common mechanism: A **Session Token** (a string of random characters) that is generated and placed in a **Cookie**.

Authentication (3)

A **Cookie** is a set of data stored in a web client (such as a browser) and associated with a specific URI.

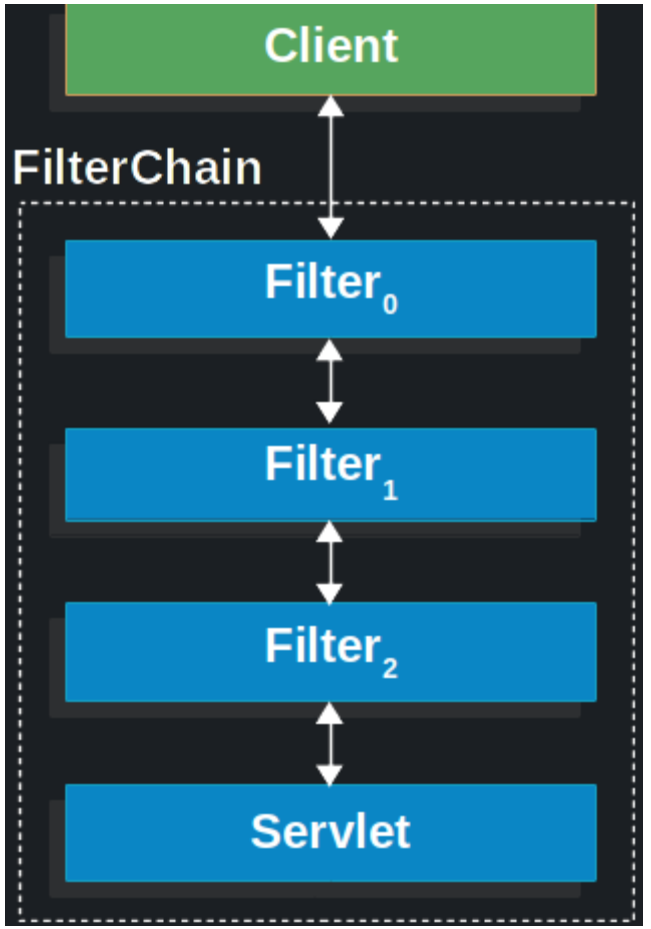
- ***Cookies are automatically sent to the server with every request*** (no extra code needs to be written for this to happen).
- As long as the server checks that the Token in the Cookie is valid, unauthenticated requests can be rejected.
- Cookies can persist for a certain amount of time even if the web page is closed and later re-visited. This ability typically improves the user experience of the web site.

Spring Security and Authentication

Spring Security implements authentication as a Filter Chain.

The Filter Chain is a component of Java web architecture which allows programmers to define a sequence of methods that get called prior to the Controller. Each filter in the chain decides whether to allow request processing to continue, or not.

Spring Security inserts a filter which checks the user's authentication and returns with a 401 UNAUTHORIZED response if the request is not authenticated.



Authorization

- Up until now we've discussed authentication. But in reality, authentication is only the first step. **Authorization** happens after authentication and allows different users of the same system to have different permissions.
- Spring Security provides Authorization via **Role-Based Access Control (RBAC)**. This means that any Principal can be associated to a number of **Roles**. Each resource (or operation) specifies which Roles a Principal must have in order to perform actions with proper authorization.
- For example, a user with an **Administrator Role** is likely to be authorized to perform more actions than a user with a **Customer Role**. In Spring, You can configure role-based authorization at both a global level and a per-method basis.

Example

- User: View products, place orders
- Product Owner: Add and update products
- Administrator: Manage users, products, and system settings

Global Authorization: You can secure URL patterns across the whole application.

.requestMatchers("/admin/**").hasRole("ADMIN")

.requestMatchers("/products/**").hasRole("PRODUCT_OWNER")

Same Origin Policy

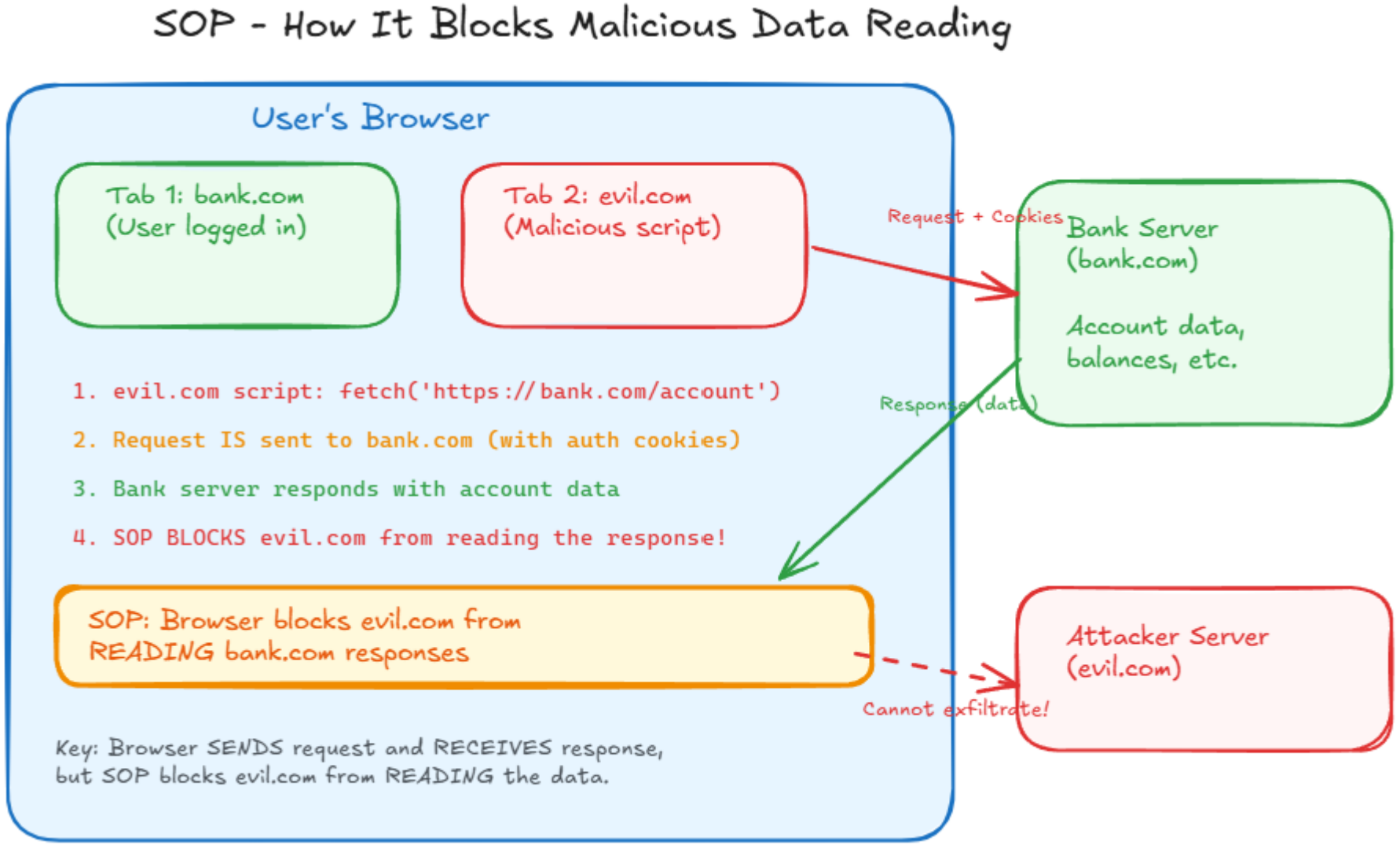
- The most basic mechanism of protection relies on HTTP clients and servers implementing the **Same Origin Policy (SOP)**: ***The Same Origin Policy (SOP) prevents a script running on one origin from reading data returned by another origin*** **(an origin is defined by a protocol, domain, and port)*.***
- ***Browsers enforce the Same Origin Policy to prevent one website from reading data returned by another website.***

- SOP is critical to the security of web sites because without it, any web page could contain a script that reads sensitive data from another site. For example, a malicious page could use JavaScript to fetch your bank's account summary page and read your balance, transaction history, and account numbers, all without your knowledge.

•**Note:** SOP does **not** prevent cross-origin requests from being *sent,* it prevents the response from being *read*. This means SOP alone does not protect against **Cross-Site Request Forgery (CSRF)**, where an attacker tricks the browser into submitting a state-changing request (like a funds transfer) to another site. CSRF is mitigated by other mechanisms such as **CSRF tokens and SameSite cookie attributes**.

Same Origin Policy

•For example, suppose a user is logged into his bank website and visits a malicious web page in another tab. Without SOP, the malicious page could send a request to the banking site (which would include the user's auth cookies), **read the response**, and exfiltrate sensitive data like account balances, account numbers, and transaction history to the attacker's server.



As a user: choose your browser with caution: never use untrusted browser

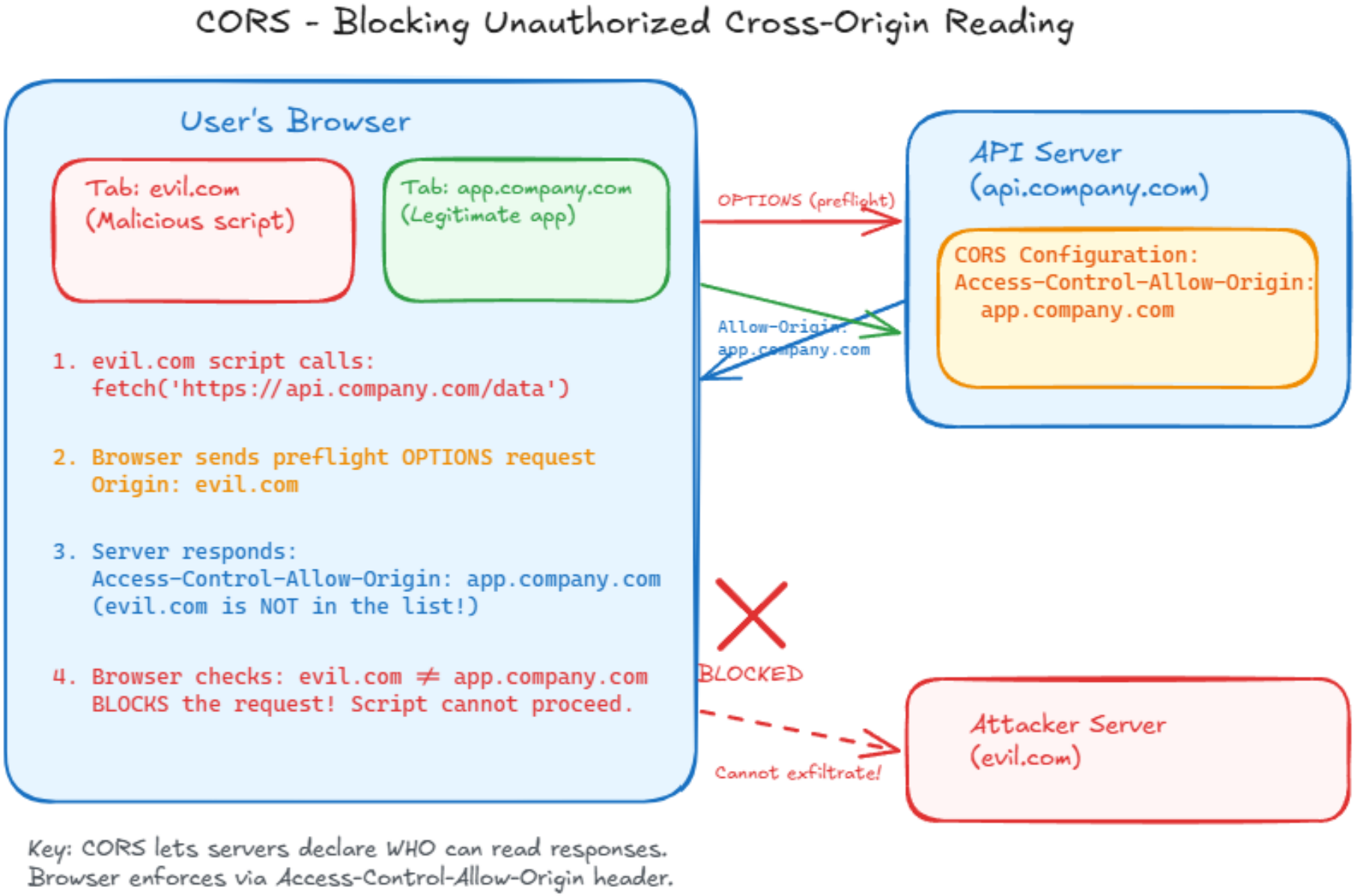
Cross-Origin Resource Sharing CORS

- Sometimes a system consists of services running on several machines with different URIs (i.e. Microservices architecture).
- Cross-Origin Resource Sharing (CORS) is a way that browsers and servers can cooperate to relax the SOP.
- A server can explicitly specify a list of “allowed origins” that are allowed to **read the responses**. **The server declares its CORS policy via HTTP response headers (like Access-Control-Allow-Origin), and the browser enforces it**
- Spring Web provides the **@CrossOrigin** annotation, allowing you to specify a list of allowed sites. @CrossOrigin(origins = "http://localhost:3000")
- If you use the annotation without any arguments, it will allow all origins, which is a bad practice.
- the browser sends **preflight requests** (OPTIONS HTTP method) before certain cross-origin requests to check if the server allows them.

Common Web Exploits

- Along with exploiting known security vulnerabilities, malicious actors on the web are also constantly discovering new vulnerabilities.
- Thankfully, Spring Security provides a powerful tool set to guard against common security exploits. Let’s discuss two common exploits, how they work, and how Spring Security helps to mitigate them.

Cross-Origin Resource Sharing CORS

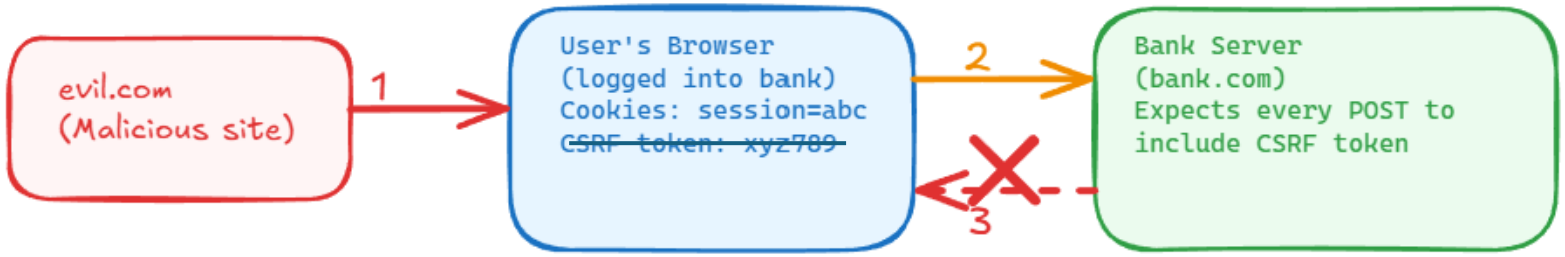


1. evil.com's script calls fetch('https://api.company.com/data')
2. Browser sends a **preflight OPTIONS request** with Origin: evil.com
3. Server responds with Access-Control-Allow-Origin: app.company.com, evil.com is **NOT** in the list
4. Browser checks: evil.com ≠ app.company.com : **BLOCKS** the request. Script cannot proceed.

Cross-Site Request Forgery CSRF

- One type of vulnerability is a **Cross-Site Request Forgery (CSRF)** which is often pronounced **“Sea-Surf”** and also known as **Session Riding**.
- CSRF is possible because browsers automatically include cookies (including session/auth cookies) with every request to a domain. CSRF attacks happen when a malicious piece of code sends a request to a server where a user is already authenticated. When the server receives the Authentication Cookie, it has no way of knowing if the victim sent the harmful request unintentionally.
- To protect against CSRF attacks, you can use a **CSRF Token**. A CSRF Token is different from an Auth Token as it is not stored in a cookie and therefore not automatically sent by the browser.
- The server generates a token and embeds it in the page (e.g., as a hidden form field). When the form is submitted, the server checks that the token matches. Since **the attacker cannot read the token from another origin (SOP prevents this)**, they can't forge a valid request.
- Spring Security has built-in support for **CSRF tokens which is enabled by default.**

Cross-Site Request Forgery CSRF



Attack Flow (blocked by CSRF token)

Step 1: User visits evil.com. A hidden form auto-submits:

```
<form action='https://bank.com/transfer' method='POST'>
  <input name='to' value='attacker'>
  <input name='amount' value='10000'>
</form>
```

Step 2: Browser sends POST to bank.com with cookies (auto-attached).
BUT evil.com cannot read the CSRF token (SOP blocks this!),
so the request has NO valid CSRF token.

Step 3: Bank server checks: CSRF token missing or invalid → REJECTS!
The forged transfer is blocked. User's money is safe.

Why it works: The CSRF token is embedded in bank.com's page.
SOP prevents evil.com from reading it. Attacker can trigger a request but can't include tl

Cross-Site Scripting XSS

- Even more dangerous than CSRF vulnerability is **Cross-Site Scripting (XSS).** This occurs when an attacker is somehow able to “trick” the victim application into executing arbitrary code. There are many ways to do this. A simple example is saving a string in a database containing a <script> tag, and then waiting until the string is rendered on a web page, resulting in the script being executed.
- XSS is potentially more dangerous than CSRF.
- XSS executes code **on the client (browser) only.** XSS attacks depend on security “holes” caused by poor programming practices.
- XSS can bypass CSRF protections.
- If an attacker achieves XSS on bank.com, the injected script runs *on bank.com's origin*, meaning it can read CSRF tokens, session cookies (if not HttpOnly)
- The main way to guard against XSS attacks is to **properly process all data from external sources** (like web forms and URI query strings). In the case of our <script> tag example, attacks can be mitigated by properly escaping the special HTML characters when the string is rendered.
- Content Security Policy (**CSP**) headers**, HttpOnly** cookies (to prevent cookie theft via XSS), and **input validation** are all important defenses.

Securing a REST API with JWT

- Like any server application, a Spring Boot REST API has to be secured**. Spring Security and JWT** are a good combination for implementing server security.
- The following includes:
  - Understanding the basics of authentication, authorization, and web application defence
  - Identifying and evaluating authentication and authorization options
  - Securing a REST API using JWT authentication
  - Securing Spring MVC endpoints and Spring Data queries using Spring Security primitives

What is **Spring Security?**

- Spring has already security-first defaults that implement security features. It also allows customization.
- Spring Security is a powerful package for securing Spring Boot API with JWT.

Problems with our API from chapter 6

Neglecting to secure our API's endpoints has at least three consequences:

- The content is ***public***, you can't control who sees the information
- The content is ***anonymous***, you can't know who is asking
- The content is ***unprotected***, bad actors can take advantage of Browser-based vulnerabilities.

Public Content

- Because the content is public, any user with access to the network location can command the API and see the data. While this can be mitigated somewhat with network security, practically speaking, most REST APIs are often exposed to the public internet through browsers or API gateways.
- Even if they weren't exposed to the public internet, the threat of server-side request forgery (SSRF) is real and we should not leave any of our production APIs open like this.
- **Server-side request forgery** is a web security vulnerability that allows an attacker to cause the server-side application to make requests to an unintended location (such as another server within the organization).

Anonymous Content

- Because the content is anonymous, we can't decide if the user is known, trustworthy, and authorized.
- Practically speaking, we cannot only allow the user to access user's specific content because his identifier is not in the request.

- This can be alleviated by requiring authentication.

Unprotected Content

Because the content is unprotected, when this REST API is exposed to a browser, it may make the application as a whole vulnerable to CSRF(see-surf), MITM, XSS and other attacks without additional intervention.

In security terms, this application (our example from chapter 6) can neither *authenticate* nor *authorize* requests and it cannot mitigate common security vulnerabilities.

- CSRF: Cross-Site Request Forgery is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated.
- MITM: The Manipulator-in-the middle attack intercepts a communication between two systems and acts as a proxy between them.
- XSS: Cross-Site Scripting attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user.

https://owasp.org/www-community/attacks/csrf

About 404s

When the user requests /a-non-existent-endpoint, he should see a 404 Not Found error:

```
{
  "error": "Not Found",
  "path": "/a-non-existent-endpoint",
  "status": 404,
  "timestamp": "..."
}
```

While a 404 Not Found isn't a bad thing, when left unsecured like this, it can lead to **Fingerprinting**, **Directory Traversal** and other enumeration-like vulnerabilities.

Secure Defaults

- ***Secure by Default*** is a **principle** that encourages you to ensure that your *default settings are secure*. In this way, if an application finds itself in production with the defaults, it isn't a security vulnerability.
- ***Principle of Least Privilege*** is a **principle** that encourages you to think about each piece of data as a privilege to possess and to *give end users the lowest privileges possible* that let them do their work well.
- Spring Security embraces both of these philosophies to automatically secure REST APIs

Spring Security Defaults

When Spring Security is on the classpath (or Maven dependencies), Spring Boot works to configure your application with the following defaults for a REST API:

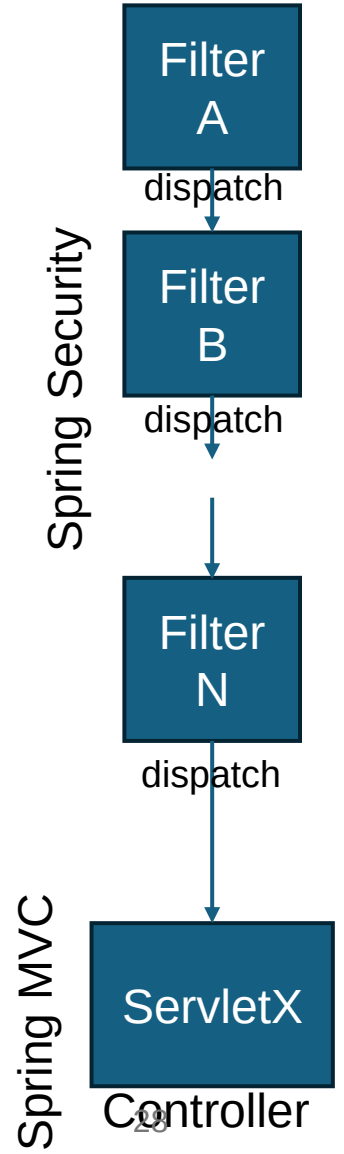
- Requires authentication for all requests
- Responds with secure headers for all requests
- Requires CSRF mitigation for all requests with side-effects (post, put, delete)
- Allows for HTTP Basic authentication with a default user
- Responds RESTfully to security failures (responds with appropriate status codes)
- Protects against malicious requests with an application firewall

..Requires Authentication for All Requests

- Whether the endpoint is generated by the developer or by Spring Boot, all requests across all dispatches require authentication.
- Regardless of the nature of the endpoint, Spring Security applies a Filter chain that inspects every request and rejects it if the request is unauthenticated.
- This is one of Spring Security's secure defaults.

Servlets, Filters, and Dispatchers

- **The Java Servlet API** is the module in Java for processing HTTP requests inside of an application. Using servlet terminology, a given HTTP request can pass through multiple *dispatches*. Each dispatch can be intercepted by multiple *filters* on its way to a single *servlet*.
- A ***servlet*** handles HTTP requests and yields an HTTP response. You can think of a servlet as a "mini-server".
- A ***filter*** intercepts HTTP requests to handle cross-cutting concerns. Typically, filters either enrich the request in some way or they deny the request, preventing it from reaching the servlet.
- A ***dispatch*** represents a single pass an HTTP request makes through a set of filters and its target servlet. Typically, an HTTP request first passes through the REQUEST dispatch, but can also subsequently pass through the ERROR dispatch, the FORWARD dispatch, and others.
- In Spring terms, Spring MVC constitutes a single servlet, Spring Security constitutes a set of filters, and Spring Boot ships with an embedded container that performs the various dispatches needed to service a single request.
- All of this means that Spring Security defaults require that every *dispatch* be authenticated.



Security Benefits

- The nice thing about this arrangement is that it doesn't matter who created the endpoint. If it is you, Spring Boot, or a third-party, Spring Security's servlet filter intercepts the request before any servlet (controller) can process it.
- This means that **when you include Spring Security even non-existent endpoints will return a 401 Unauthorized HTTP response status code** instead of, say, a 404 Not Found, the default Spring Boot response for non-existent endpoints.
- The reason for this strict policy is because of the **Principle of Least Privilege**. This principle says that you should offer only the information that the end user is privileged to know.

Security Benefits

So what's the big deal? What's so privileged about a non-existent endpoint (404)?

For security purposes, even such URIs (404) are a privileged information. You can imagine if someone requested index.jsp or /admin. If Spring Security returned a 404 in those cases instead of a 401, that would mean 404 is a hint to a bad actor that a given endpoint does not exist! The bad actor can use this hint to enumerate the REST APIs valid URIs, figure out underlying vulnerable technologies, and accelerate their attack.

…Responds With Secure Headers for All Requests

HTTP headers allow a client and server to exchange additional information between each other in an HTTP request and response. Whether a request is authenticated or not, Spring Security always responds with certain headers by default. Each header defaults to the most secure value available.

Caching Headers

- The first are cache control headers. One class of browser-based vulnerabilities is that HTTP responses get cached in the browser. For example, suppose your REST API returned the following: `[ { "amount": 123.45, "id": 99, "owner": "sarah1" } ]`

Then that response could be cached in the browser for later retrieval by a bad actor on the user's local machine. Or, more practically, even if an end user loses access themselves or revokes it from the client application, the browser may still be able to retrieve that sensitive data from its cache.

- Spring Security applies secure settings for ***Cache-Control*** **and other headers** to mitigate this class of vulnerabilities.

Responds With Secure Headers for All Requests (2)

without any additional configuration, Spring Security automatically adds several HTTP security headers to every response sent from your application. These headers help protect against common web vulnerabilities.

The default secure headers that Spring Security adds include:

1. **X-Content-Type-Options: nosniff** - Prevents browsers from MIME-sniffing (multipurpose internet mail extension) a response away from the declared content type
2. **X-Frame-Options: DENY** - Prevents your page from being placed in an iframe, protecting against clickjacking attacks
3. **X-XSS-Protection: 1; mode=block** - Enables browser's built-in XSS filtering capabilities
4. **Cache-Control: no-cache, no-store, max-age=0, must-revalidate** - Prevents sensitive information from being cached
5. **Pragma: no-cache** - Legacy header for HTTP 1.0 clients, also prevents caching
6. **Expires: 0** - Another cache control header
7. **Strict-Transport-Security** - Ensures connections to the site are always using HTTPS (when HTTPS is enabled)
8. **Content-Security-Policy** - Helps prevent various types of attacks including XSS and data injection attacks. Controls what resources the browser is allowed to load. Only load and execute content from trusted sources

Content Type Options

- X-Content-Type-Options header: This header tells browsers to not try to guess the content type of a response.
- A common place where bad actors hide is where the HTTP protocol is fuzzy and applications try to understand, disambiguate, and guess the intent of the request or response.
- A browser, for example, may look at a response that starts with <html> and reasonably guess that the content type is text/html – that is, a web page. Sometimes this guessing is unsafe. For example, it is possible for an image to contain scripting content and the browser can be tricked into guessing and executing steal-my-password.jpg as JavaScript. Unsafe!
- Spring Security addresses this by issuing a secure setting for X-Content-Type-Options by default.

Responds With Secure Headers for All Requests (3): **X-Content-Type-Options: nosniff**

MIME-type (media types: audio, video, etc) sniffing is a behavior in web browsers where they try to guess the type of content being served, even if the server explicitly declares it. This can be dangerous because:

- A file intended to be harmless (like a .txt) could be interpreted as executable code.
- This HTTP header tells the browser: **"Trust the server's declared content type. Don't guess or 'sniff' it."**
- So if the server says a file is text/plain, the browser must treat it strictly as plain text even if it looks like HTML or JavaScript.
- A **drive-by download attack** happens when a user unknowingly downloads and executes malicious content just by visiting a webpage.
- Without nosniff, an attacker could:
  - Upload a malicious script disguised as a .jpg or .txt
  - Trick the browser into executing it as JavaScript
  - Exploit the user's browser or system

Responds With Secure Headers for All Requests (4)

What Is Clickjacking? **Clickjacking is a malicious technique where an attacker:**

- Loads your legitimate site inside an invisible or disguised iframe.
- Places deceptive content (like fake buttons or forms) on top of it.
- Tricks users into clicking — thinking they're interacting with the attacker's page, but they're actually clicking on your site underneath.

This can lead to:

- Unintended actions (e.g., liking a post, submitting a form)
- Disclosure of sensitive information
- Security breaches if the user is authenticated

By setting: **X-Frame-Options: DENY**

- You instruct browsers to **never** render your site inside a frame, not even on your own domain. This completely blocks iframe embedding and neutralizes clickjacking attempts.

Responds With Secure Headers for All Requests: **Strict Transport Security Header**

- This header forces a browser to upgrade requests to HTTPS for a specified period of time.
- ***NOTE:*** Since this is intended for HTTPS requests, it isn't written by default for an HTTP request. Given that, you might not see it in your local testing over HTTP.
- HTTPS has long been shown to be a critical component of secure deployments. Man-in-the-middle attacks make it possible for the data passing between the end user and the REST API to be viewed and modified.
- Such attacks are mitigated by HTTPS, and the Strict Transport Security header tells the browser to not send any requests to this REST API over HTTP. Instead, any HTTP requests should be automatically upgraded by the browser to HTTPS.

…Requires CSRF Mitigation for All Requests With Side-Effects

- Another place where REST APIs are at risk is the ability for third-party web sites to make requests to them without the user's consent.
- This is possible **since browsers, by default, send all cookies and HTTP Basic authentication details automatically to any endpoint.**
- For example, take a look at this request for a so-called image

```
<img src="https://mybank.example.org/account/32?transfer=25&toAccount=45" />
```

- Yikes! This request will be executed by the browser. This works because the browser has no way of knowing whether the URL points to an image until the response comes back. By then, the damage has already been done.
- As you can imagine, browsers even make this request on third-party websites. Browsers, by default, will send all of mybank.example.org's cookies and HTTP Basic credentials to it by default as well. This means that if your user is logged in, a third-party application can command your REST API without further protection.
- Because of that, Spring Security automatically protects these endpoints with side-effects, like POSTs, PUTs, and DELETEs. It does this by sending a special token to the client that it should use on subsequent requests. The token is transmitted in such a way that third parties cannot see it. So when it is returned, Spring Security believes that it is legitimately from the client.

…Allows HTTP Basic Authentication With a Default User

- Spring Security generates a default user called user. Its password is generated, though, on each startup.
- The reason for that is so that if you accidentally deploy Spring Security defaults to production, no one can use the default username and password to command your application. This is another classic instance of creating an application that is Secure By Default or, in other words, an application whose default settings are secure.
- To find out the password, you can look at the Boot startup logs for this string: Using generated security password: fc7e0357-7d82-4a9c-bae7-798887f7d3b3
- The UUID in that string is the password. It will be different for each time the application starts up.
- As stated, Spring Security, by default, will accept this username and password using the HTTP Basic authentication standard, which you'll have an opportunity to practice with in just a moment.

…Responds RESTfully to Security Failures

Spring Security responds with a `401 Unauthorized` status code when credentials are wrong or missing from the request. It also, by default, will send the appropriate headers to indicate the kind of authentication that is expected. The implied meaning of a `401` is that the request is *unauthenticated*.

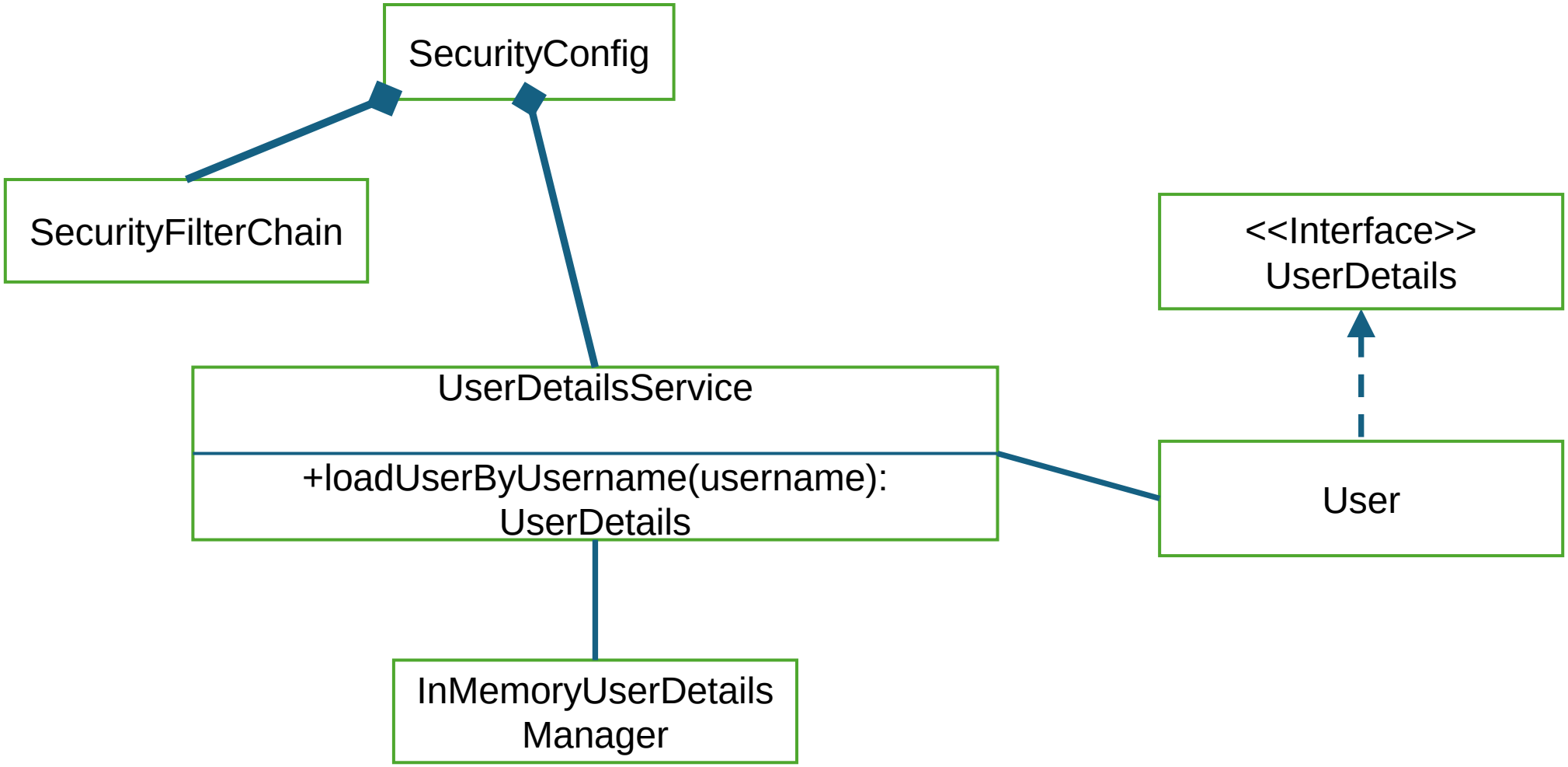
It responds with a `403 Forbidden` status code when credentials are good, but the request isn't authorized, like when an end user tries to perform an admin-only request. The implied meaning of a `403` is that the request is *unauthorized*.

…Protects Against Malicious Requests With an Application Firewall

- There are myriad other ways that a bad actor may try and misuse your REST API. With many of them, the best practice is to reject the request outright.
- Spring Security helps you with this by adding an application firewall that, by default, rejects requests that contain double encoding and several unsafe characters like carriage returns and linefeeds. Using Spring Security's firewall helps mitigate entire classes of vulnerabilities.

Application: Basic security

Architecture design (in memory DB for users)



Definition of a Java Bean

A JavaBean is a regular Java class that follows certain conventions:

- All properties are private (use getters/setters)
- A public no-argument constructor
- Implements Serializable.

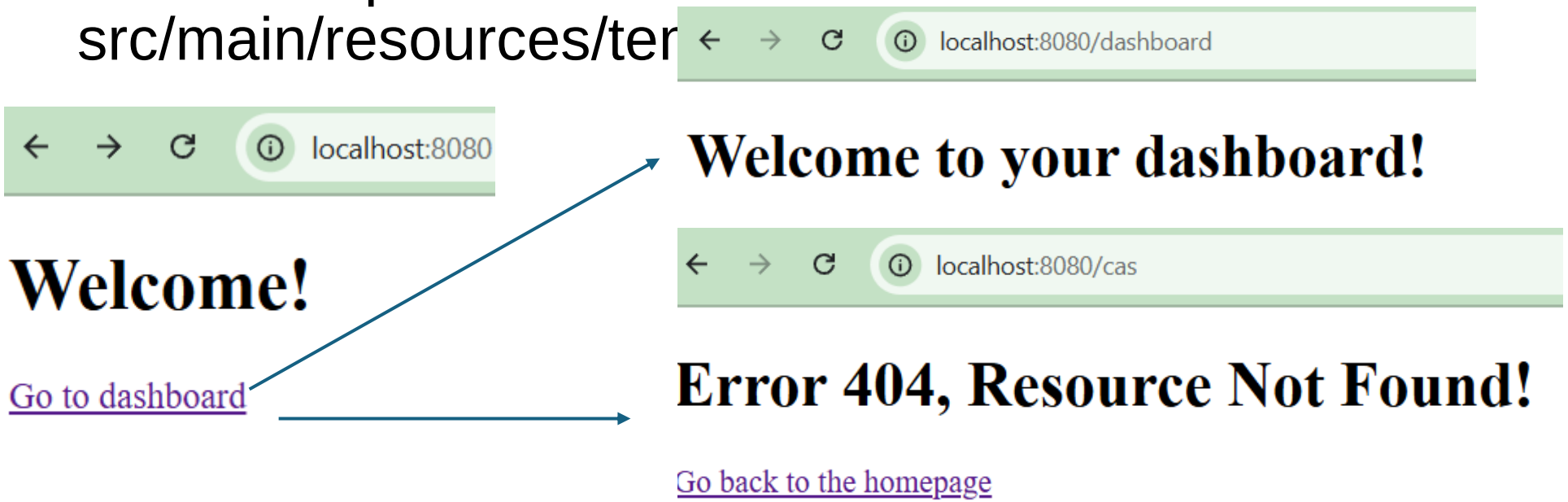
Bean methods in Spring are methods that can be used by the Spring container to create Bean objects

```
@Configuration
public class AppConfig {

    @Bean
    public MyBean myBean() {
        // instantiate, configure and return bean ...
    }
}
```

Example Application

- Our example application has three pages:
  - index.html
  - dashboard.html
  - error.html
- These templates have to be stored in src/main/resources/ter



Source code: https://github.com/skanderturki/se411/tree/master/workspace_ch_07/security

Controller to serve the pages

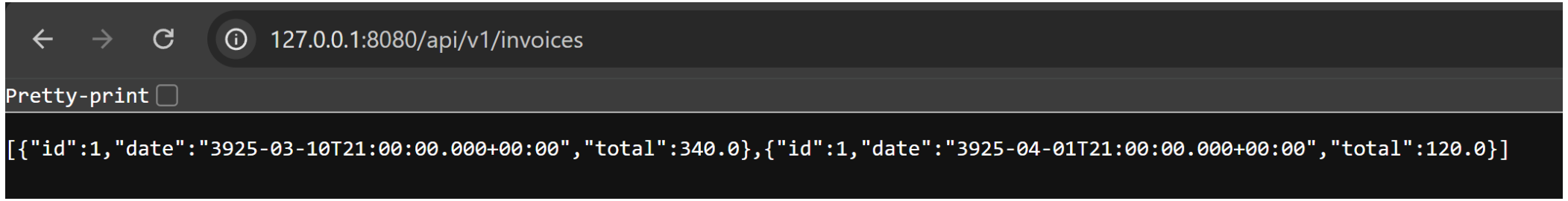
```
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;

@Controller
public class PagesController {

  @RequestMapping("/")              // This is the URI path
    public String home() {
    return "index";                 // index is the name of the template (index.html)
  }
  @RequestMapping("/login")
    public String login() {
    return "login";
  }
  @RequestMapping("/dashboard")
    public String dashboard() {
    return "dashboard";
  }
}
```

Demo: effects of simply adding the Spring Security dependency

- Download a spring boot starter code from spring initialzr with only the spring web dependency.
- Create a basic controller that returns invoices.



- Without Spring Security, no authentication required.

```
Using generated security password: df484c66-bb4a-47c6-8d92-cce7ffe00012
```

Set up Spring Security

- We want to prevent unauthorized users from viewing the dashboard page at /dashboard.
- You need to add a security check that forces the visitor to sign in before they can see that page.
- We need to configure Spring Security in the application. If Spring Security is on the classpath (with Maven, added as a dependency), Spring Boot automatically secures all HTTP endpoints with authentication.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<dependency>
  <groupId>org.thymeleaf.extras</groupId>
  <artifactId>thymeleaf-extras-springsecurity6</artifactId>
  <!-- Temporary explicit version to fix Thymeleaf bug -->
  <version>3.1.1.RELEASE</version>
</dependency>

<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-test</artifactId>
  <scope>test</scope>
</dependency>
```

- 3 dependencies: one for the application, one for Thymeleaf & Spring Security integration, and one for testing

Demo: effects of simply adding the Spring Security dependency

- Add Spring Security dependency



Add a SecurityConfig

@EnableWebSecurity:
Enables Spring Security: when applied to
a **@Configuration** class,
It activates Spring Security's web-related features.

@Bean: Indicates that a method produces a bean to be managed by the Spring container.

- **requestMatchers**("/"): Allows unrestricted access to the root URL (/).
-**anyRequest**().authenticated(): Requires authentication for all other requests.
- **.formLogin**(…) Specifies a custom login page at “/login”.
- **.logout**(…) Enables logout functionality.

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig {


 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity http)
  throws Exception {
       http.authorizeHttpRequests((requests) -> requests
              .requestMatchers("/", "/homepage").permitAll()
              .anyRequest().authenticated()
       )
       .formLogin((form) -> form.loginPage("/login").permitAll())
       .logout((logout) -> logout.permitAll());

       return http.build();
 }
```

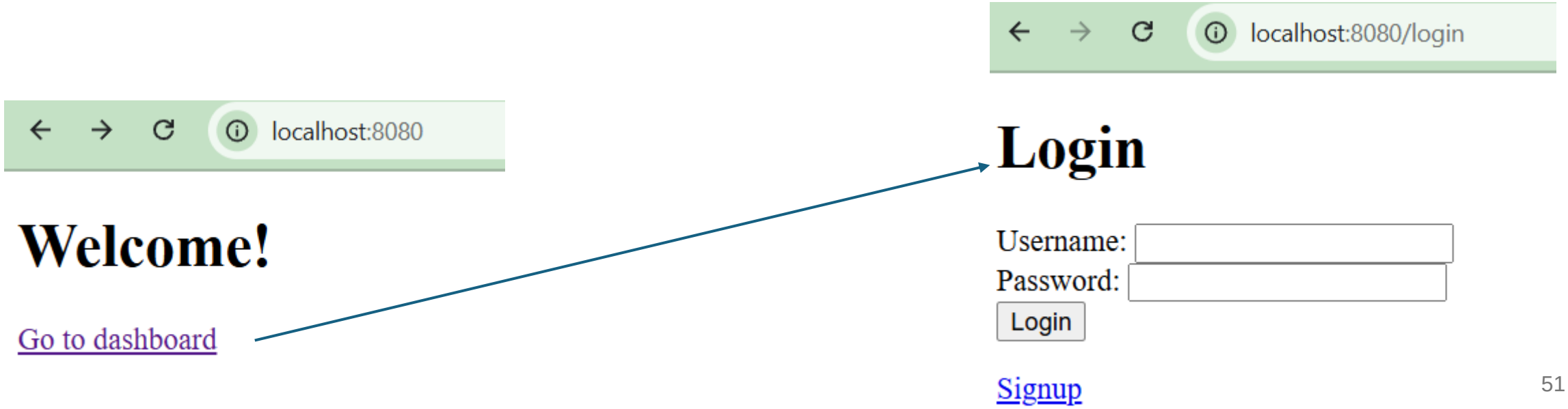
@EnableWebSecurity

@EnableWebSecurity r**egisters the Spring Security Filter Chain:**

- It sets up the necessary FilterChainProxy which routes requests through a series of security filters, handling authentication, authorization, and other security concerns.
- **Provides Basic Security Configurations:** It automatically configures fundamental aspects of web security, such as:
  - **HTTP Security:** Setting up default rules for HTTP requests, including requiring authentication for most endpoints and configuring exploit protections like CSRF.
  - **A**dds support for managing authentication processes.
  - **User Details Service and Password Encoder:** Facilitating the configuration of how user details are loaded and passwords are encoded.
- **Facilitates Customization:** While it provides default configurations, @**EnableWebSecurity** is primarily used in conjunction with a **SecurityFilterChain** bean to customize security rules and behavior.

Secured dashboard

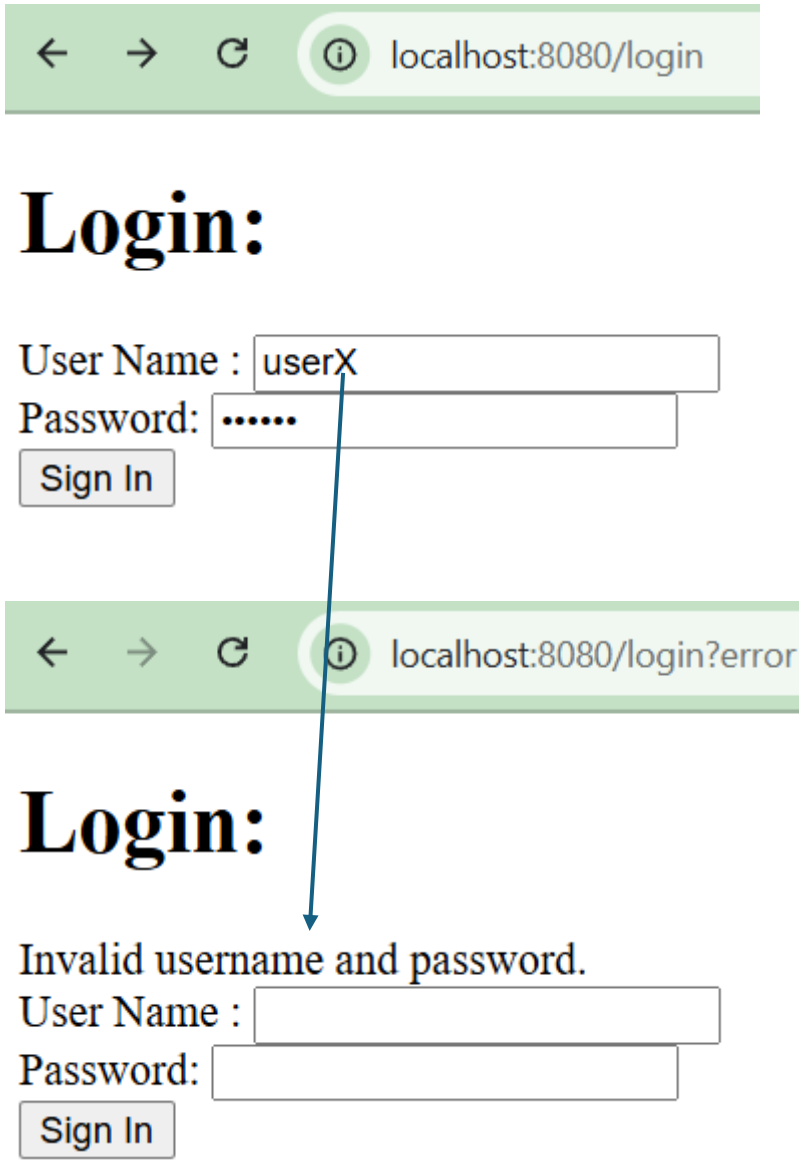
- The **WebSecurityConfig** class exposes two beans to set some specifics for the web security configuration:
- The **SecurityFilterChain** bean defines which URL paths should be secured and which should not. Specifically, the / and /homepage paths are configured to not require any authentication. All other paths must be authenticated.
- When a user successfully logs in, he is redirected to the previously requested page that required authentication. There is a custom /login page (which is specified by loginPage()), and everyone is allowed to view it.



UserDetailsService Bean

The second Bean is **UserDetailsService,** it fetches the user's details (including hashed password and roles) from a data source (e.g., a database).
In this example, the UserDetailsService bean sets up an in-memory user store with a single user. That user is given a user name of userA, a password of pwd123, and a role of USER.

```
@Bean
public UserDetailsService userDetailsService(
                              PasswordEncoder passwordEncoder) {
  UserDetails user = User.builder()
    .username("usera").password(passwordEncoder
    .encode("pwd123")).roles("USER").build();
  return new InMemoryUserDetailsManager(user);
}
@Bean
public PasswordEncoder passwordEncoder() {
   return new BCryptPasswordEncoder();
}
```



Note: creating the UI is not part of this course

The front end

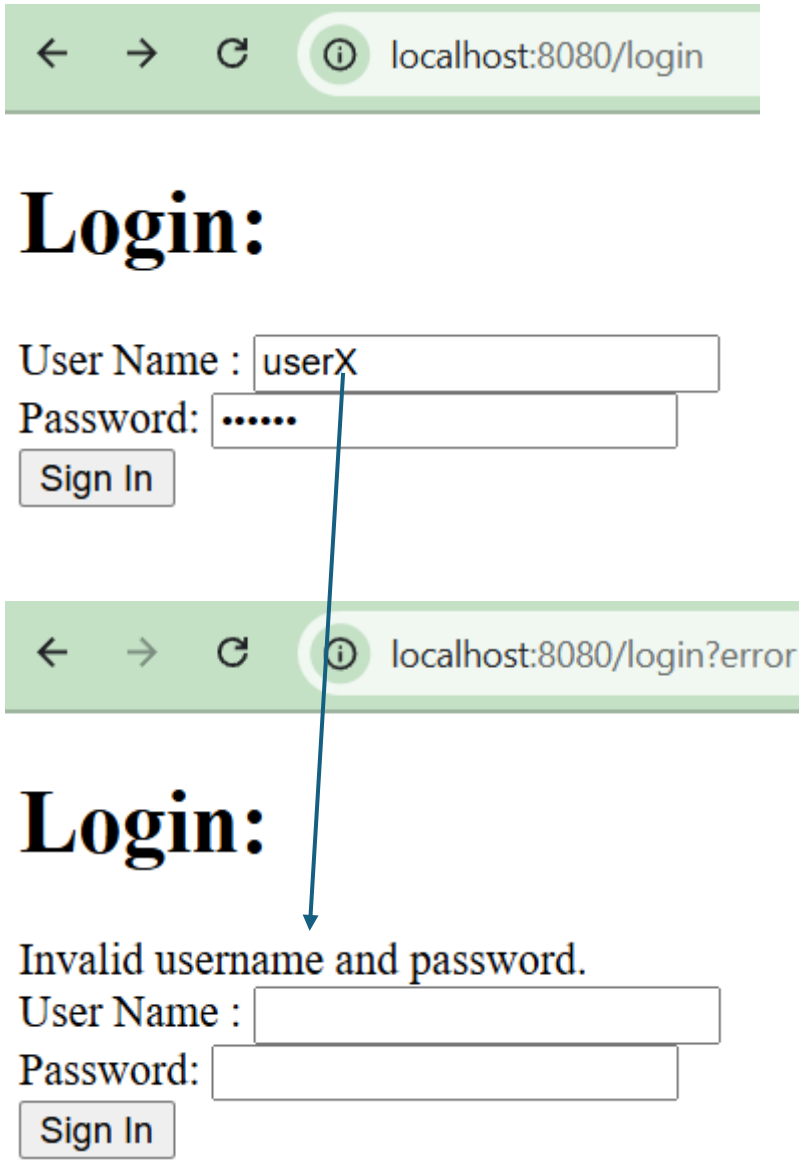
Note: creating the UI is not part of this course

```
<body>
  <div th:if="${param.error}"> Invalid username and password. </div>
  <div th:if="${param.logout}"> You have been logged out. </div>
  <form th:action="@{/login}" method="post">
     <label> User Name : <input type="text" name="username"/></label>
    <label> Password: <input type="password" name="password"/>
</label>
    <input type="submit" value="Sign In"/>
  </form>
</body>
```

Spring Security provides a filter that intercepts the login (/login) request from the form and authenticates the user.

If the user fails to authenticate, the page is redirected to /login?error, and your page displays the appropriate error message.

Upon successfully signing out, your application is sent to /login?logout, and your page displays the appropriate success message.

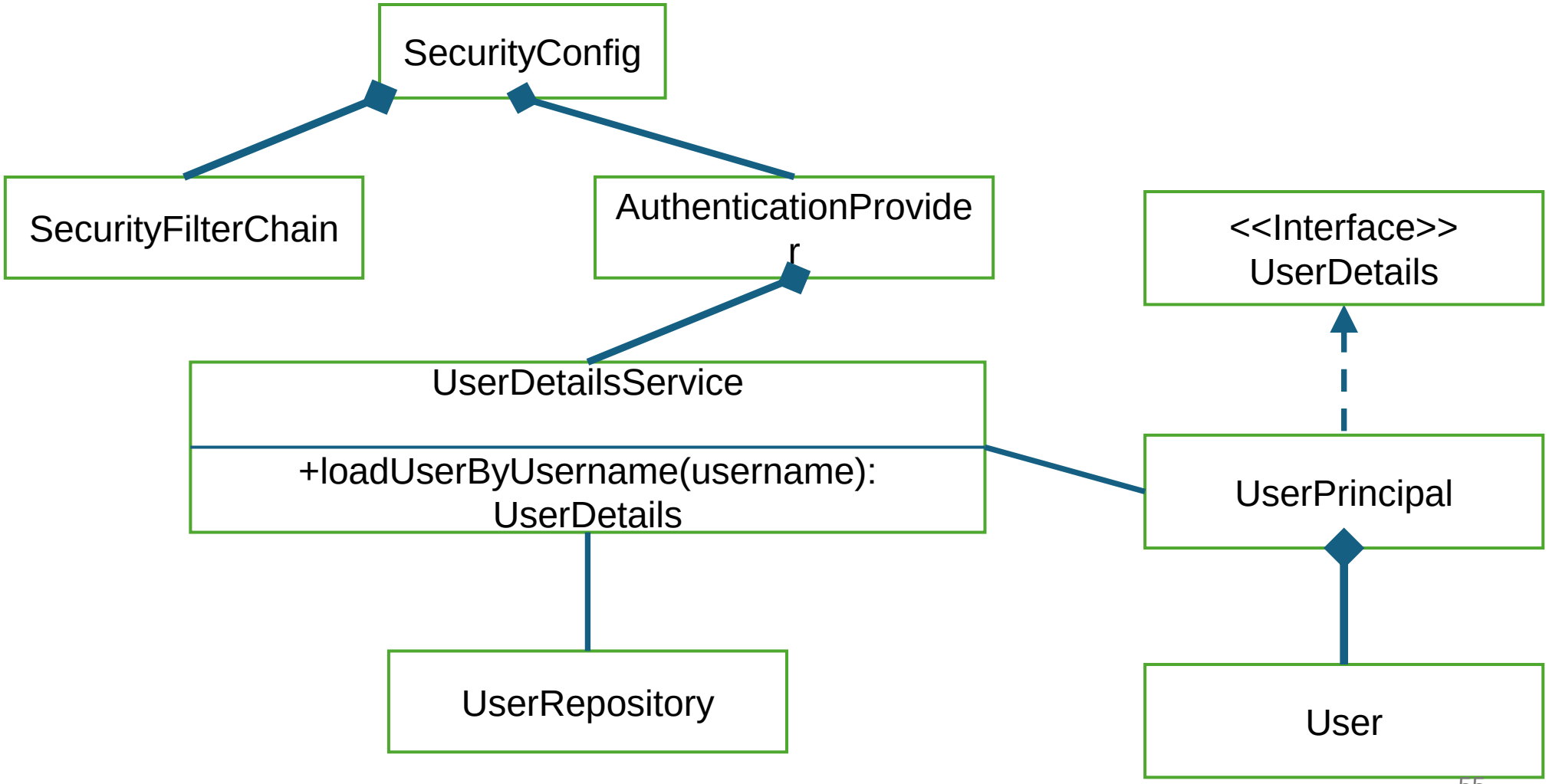


g

Implementation of basic authentication

Source code: https://github.com/skanderturki/se411/tree/master/workspace_ch_07/basiclogin

Architecture design (users stored in DB)



Reminder: the Adapter Design Pattern

```java
public class UserPrincipal implements UserDetails {

private User user;

public UserPrincipal(User user) {  this.user = user;}

@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
   return Collections.singleton(new SimpleGrantedAuthority("USER"));
}

@Override
public String getUsername() {
   return user.username();
}

@Override
public String getPassword() {
   return user.password();
}

}
```

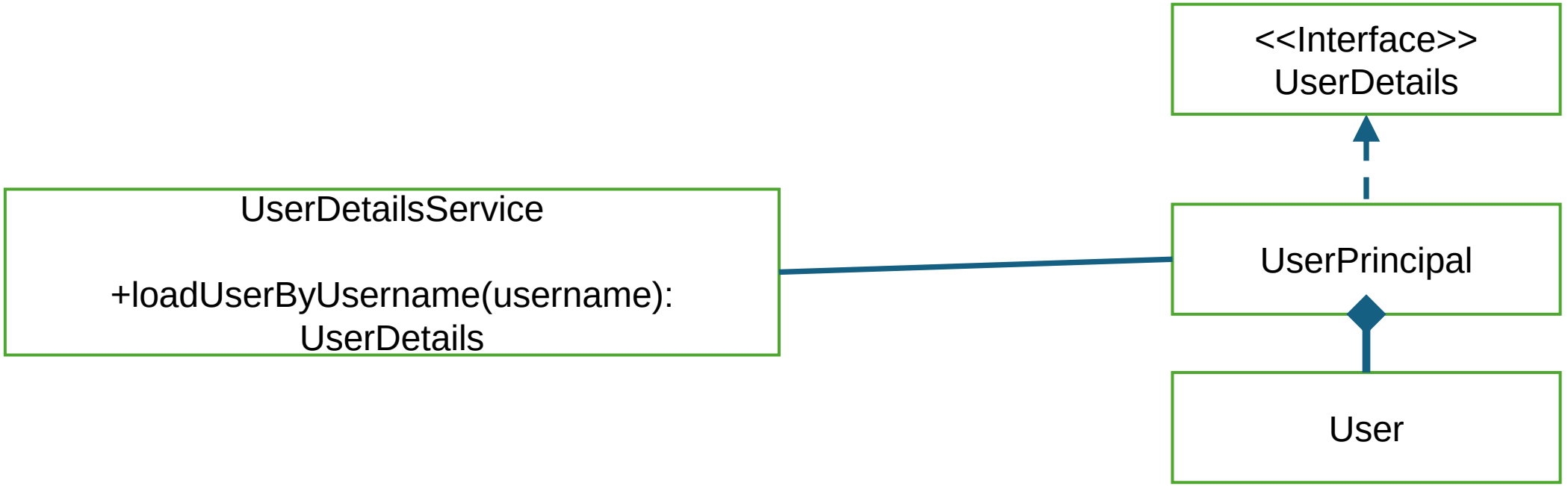
```java
public record User(
   @Id Integer id,
   String username,
   String password) {

}
```

- **Purpose:** To allow classes with incompatible interfaces to work together.
- **Mechanism:** It creates an Adapter class that acts as a bridge between a client and a service with a different interface. The Adapter implements the interface expected by the client and translates calls into the format required by the service.

UserPrincipal implements a UserDetails

- The UserDetailsService of Spring Security expects a model that corresponds to the UserDetails interface.
- In our application the User model may be different from the UserDetails interface, so we create an adapter object (UserPrincipal) that exposes to Spring Security the expected contract interface defined by UserDetails



Implementation of basic authentication

A simple Web API:

```
@RestController
@RequestMapping("/api/v1")
public class HelloController {

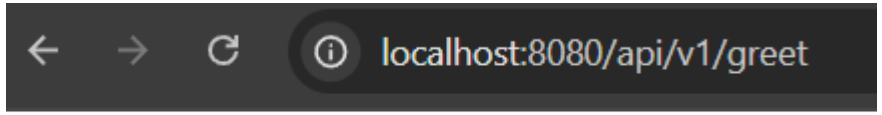
    @GetMapping("/greet")
    public String greet(HttpServletRequest request) {
        return("Selem: " + request.getSession().getId());
    }

}
```

- By adding the security packages dependency to our project, all the routes are protected:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

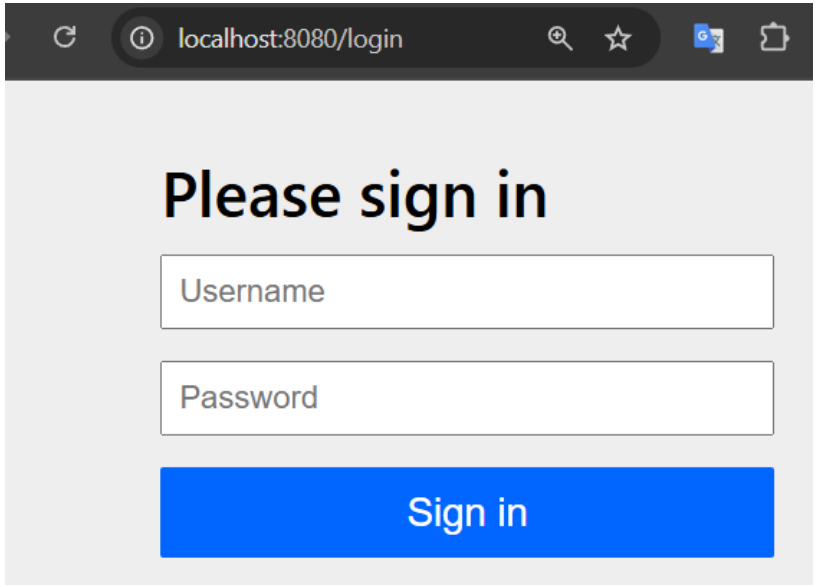
Before adding security dependencies



Selem



After adding security dependencies



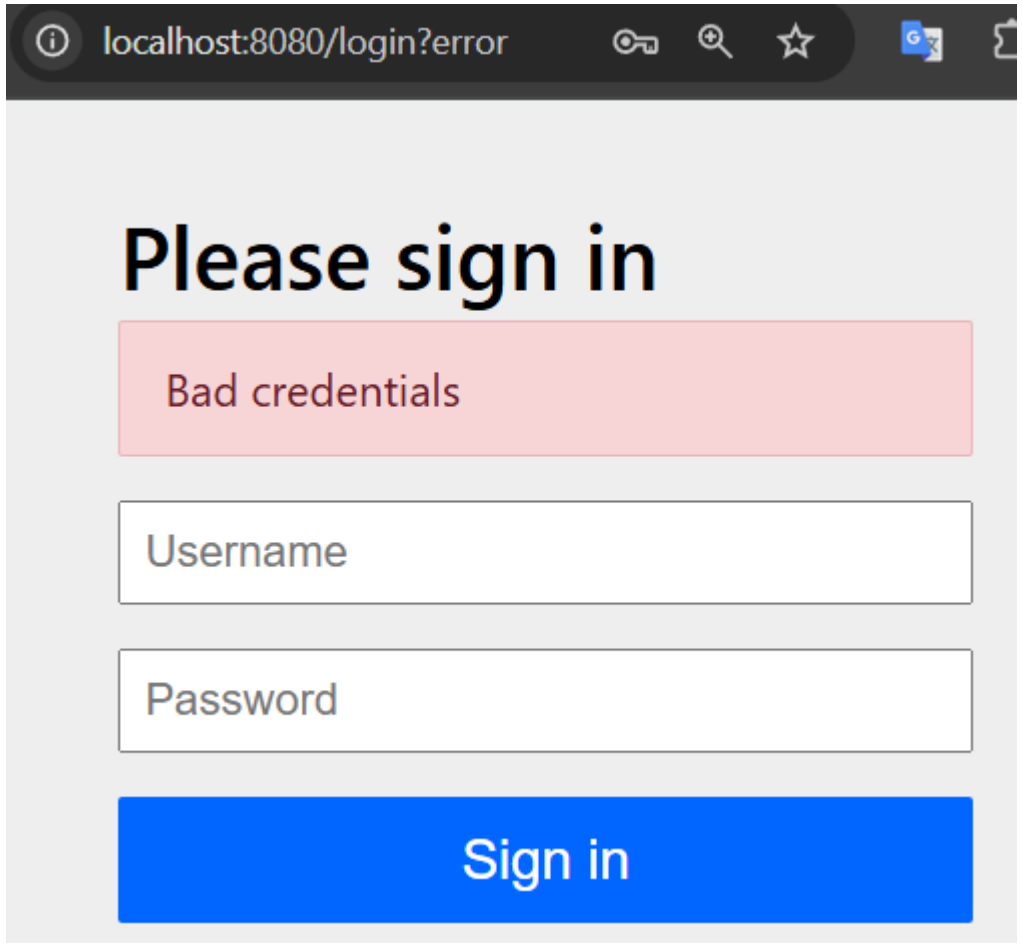
Default login / logout logic

When we start our server, we get a password generated by Spring

```
Using generated security password: 03987629-c93b-4579-973b-a1704e1325e6

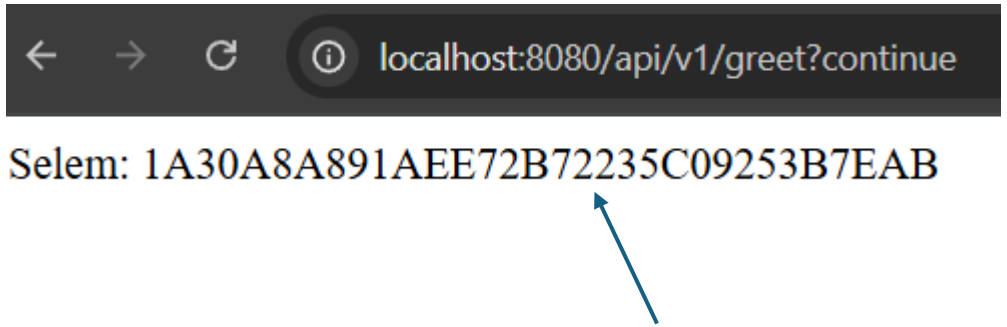
This generated password is for development use only.
```

If we use bad credentials:



If we use
User: user
Pwd: 03987629-c93b-4579-973b-a1704e1325e6

We can get the API endpoint result

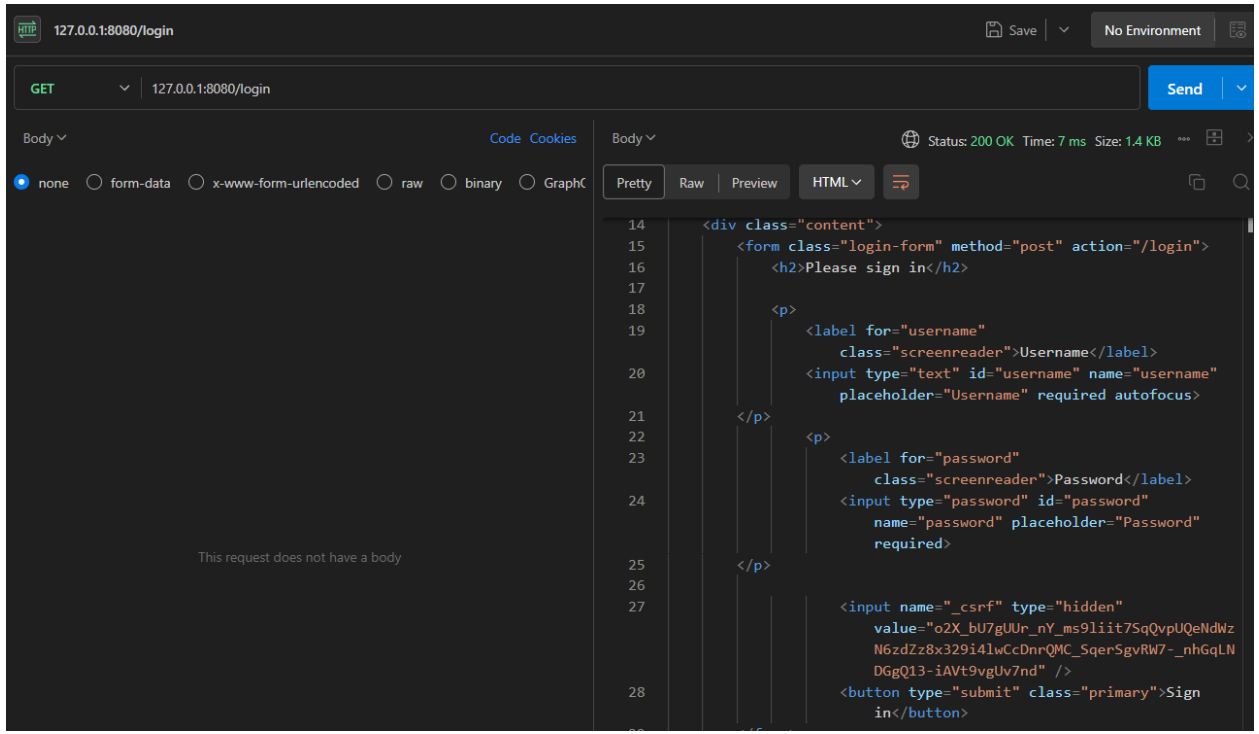


A session ID is generated for the current logged in user

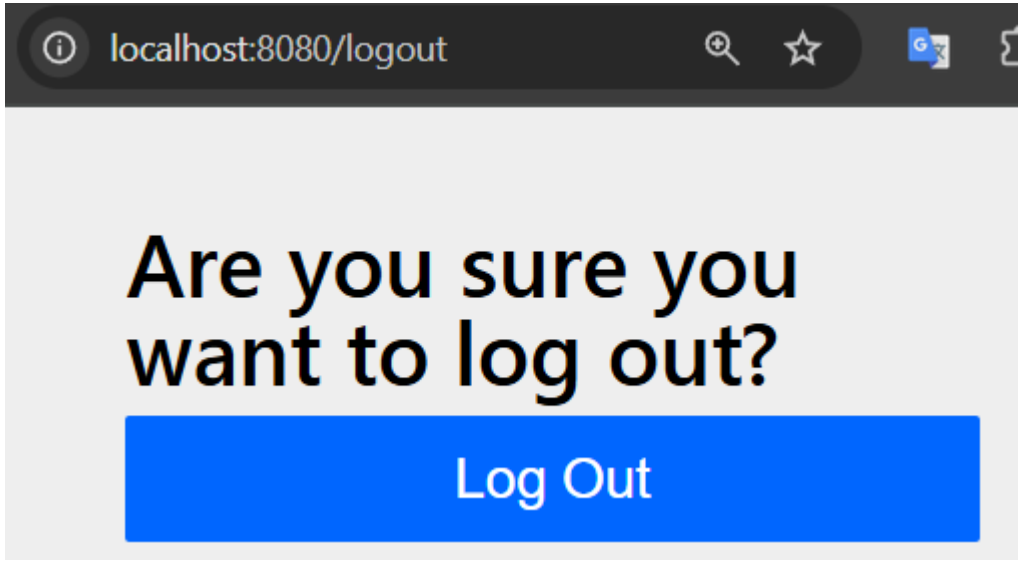
The login page carries a csrf token

The form at GET /login opens with a form an a hidden HTML input element with a _csrf token.

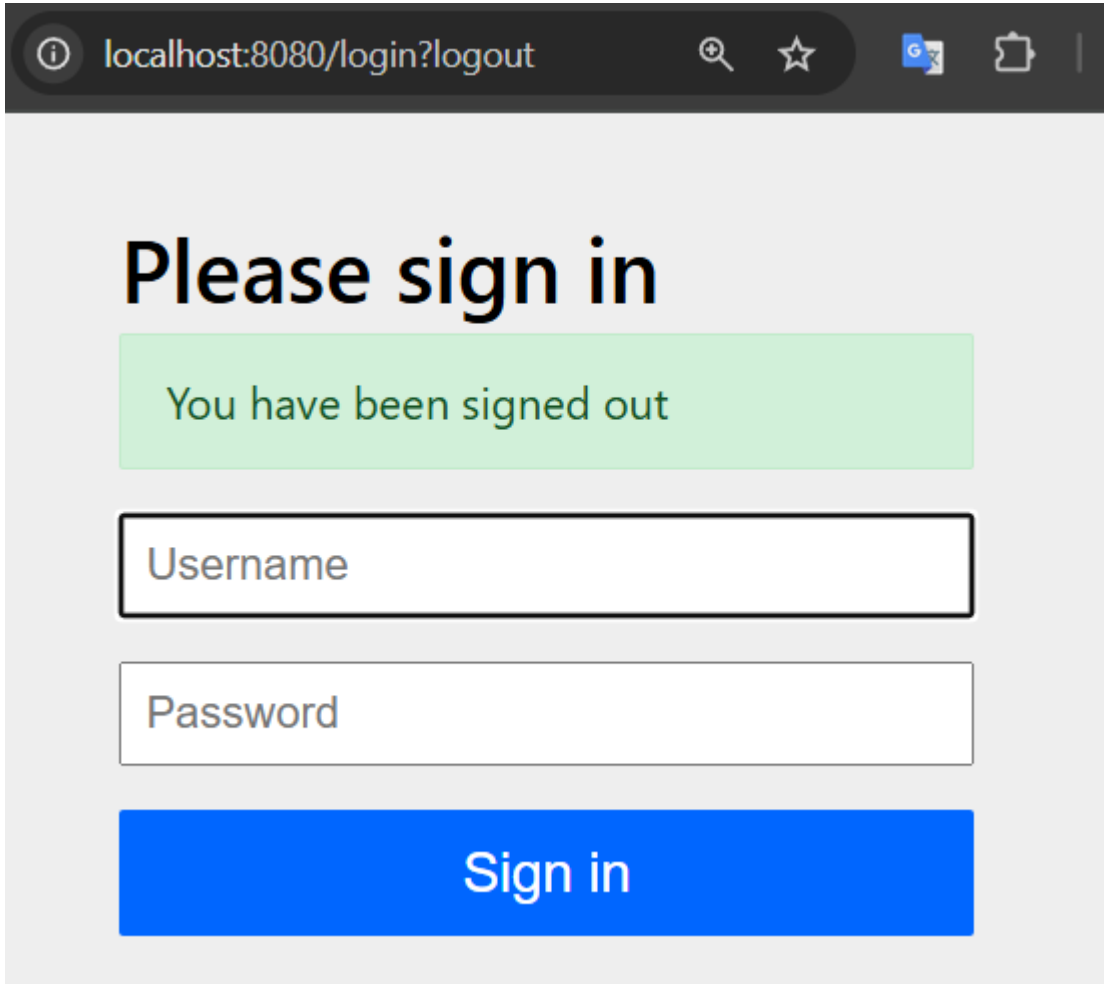
Sending the POST request to /login through the form also sends the _csrf token, this is how the server tracks if the sender or the request is still the same source.



Default login / logout logic (2)



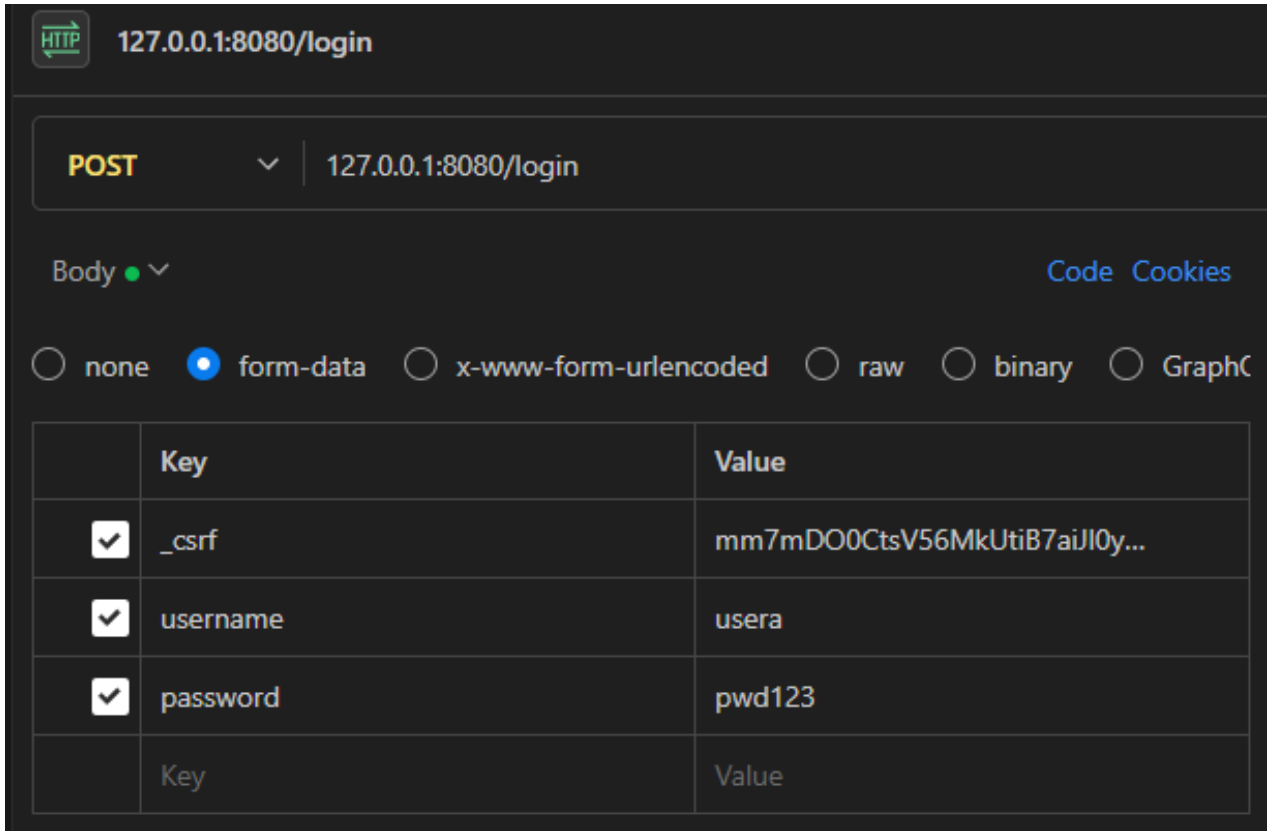


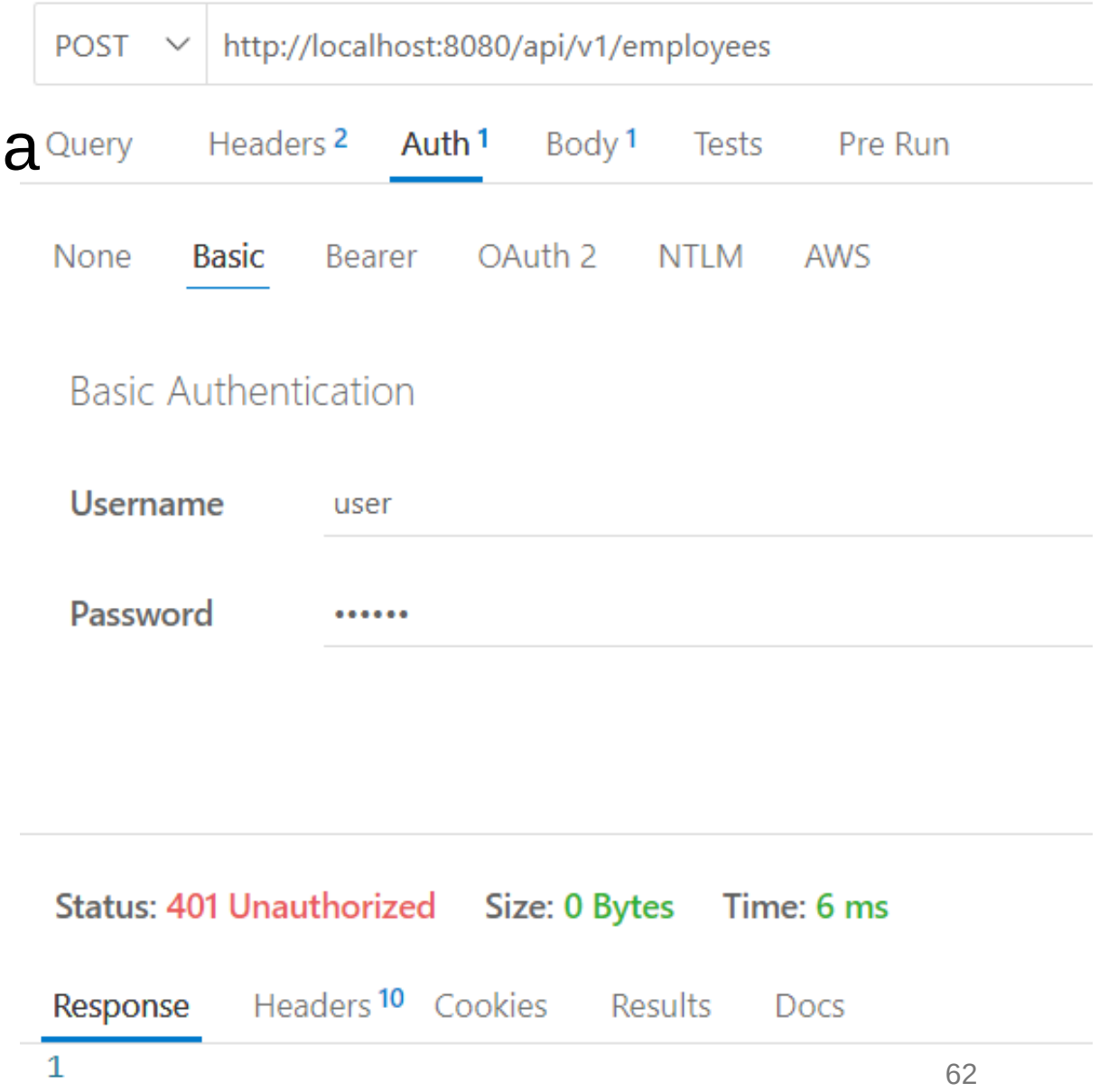


All the forms (login, logout button), the feedback messages and the logic is added automatically by Spring Security

CSRF: POST, PUT and DELETE

- With POST, PUT and DELETE, a CSRF token is required





Replacing the default filter chain

- This new filter chain will be loaded by Spring and installed as the filter chain of our server instead of the default one.
- Notice that the httpsec object is kept as is without any further definition, so our application has no filtering done currently: any user can now access any API endpoint without authentication.

```
@Configuration      // tell Spring this is a configuration class
@EnableWebSecurity  // replaces the default filter chain by the one configured here
public class SecurityConfig {


    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpsec) throws Exception{
        return httpsec.build();
    }
}
```

The new security filter chain

- We are using stateless sessions (a new session id is created for every request, the csrf token is therefore not useful anymore).
- Authentication is required for all API endpoints.
- Form authentication is not activated (as session id are recreated for every request, so the user would be redirected to relogin every time).
- Basic authentication is active (using user/pwd).

```
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity httpsec) throws Exception{
        httpsec.csrf(customizer -> customizer.disable());
        httpsec.authorizeHttpRequests(request -> request.anyRequest().authenticated());
        //httpsec.formLogin(Customizer.withDefaults());
        httpsec.httpBasic(Customizer.withDefaults());
        httpsec.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
        return httpsec.build();
    }
}
```

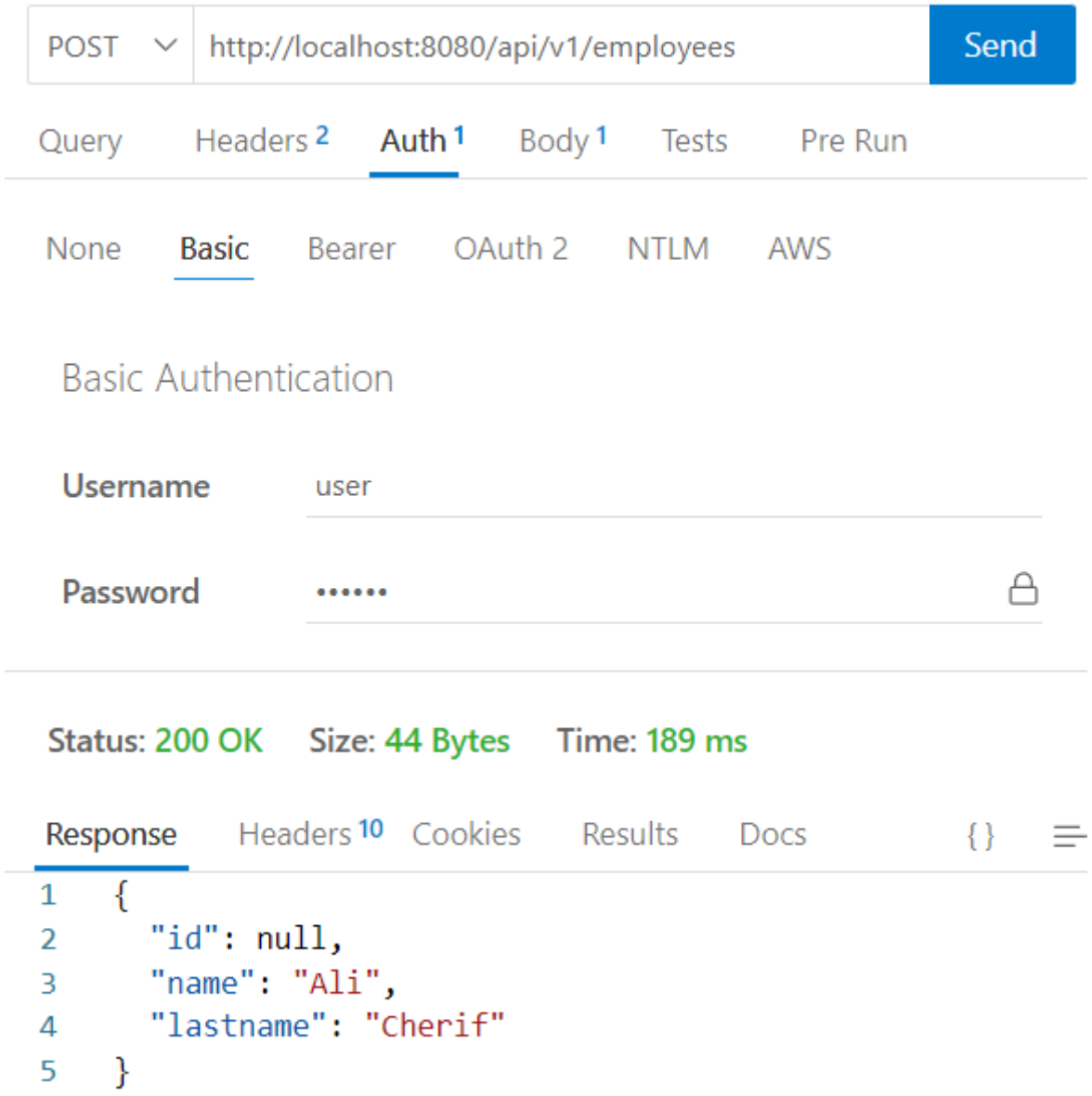
Security Filter Chain using the Builder pattern

- We can also rewrite the previous securityFilterChain method by using the Builder Pattern: chaining httpsec.func1().func2()…build()

```
@Bean // Spring will scan this method for beans
public SecurityFilterChain securityFilterChain(HttpSecurity httpsec) throws Exception {
    return httpsec.csrf(customizer -> customizer.disable())
            .authorizeHttpRequests(request -> request.anyRequest().authenticated())
            .httpBasic(Customizer.withDefaults())
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .build();
}
```

Testing POST

- By passing the username and the password, now the POST request is allowed to execute.

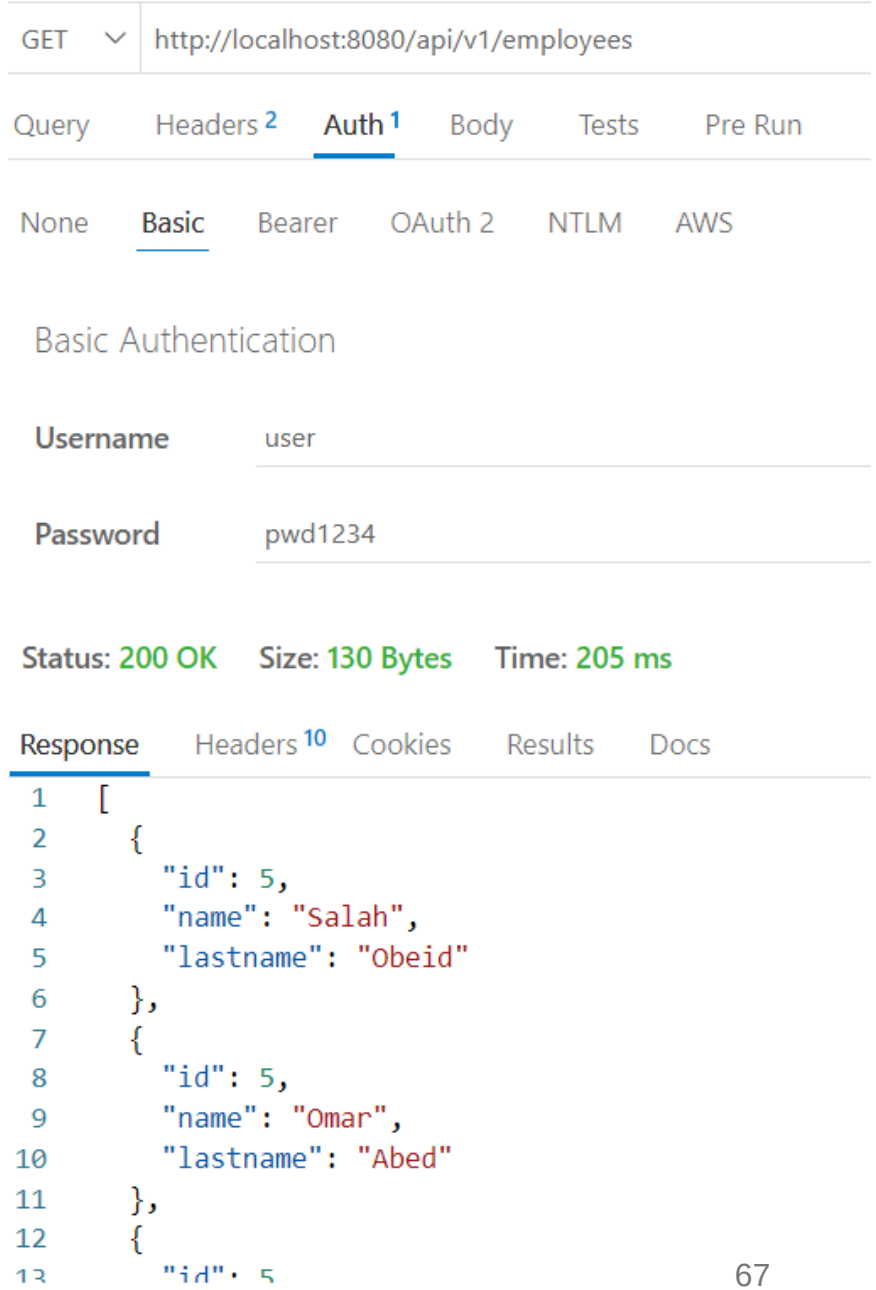


Defining users

UserDetailsService is used for retrieving a username, a password, and other attributes for authenticating with a username and password. Spring Security provides in-memory, JDBC, and caching implementations of UserDetailsService.

```
@Bean
public UserDetailsService userDetailsService() {
    UserDetails userA = User.withDefaultPasswordEncoder()
        .username("user")
        .password("pwd1234")
        .roles("USER")
        .build();

    return new InMemoryUserDetailsManager(userA);
}
```



Work with a database to store users

```java
public record User (
        @Id Integer id,
        String username,
        String password) {

}
```

```java
public interface UserRepository extends CrudRepository<User, Integer> {

    @Query("SELECT * FROM users WHERE username = :username")
    User findByUsername(@Param("username") String username);

}
```

Create a UserDetailsService that uses the DB

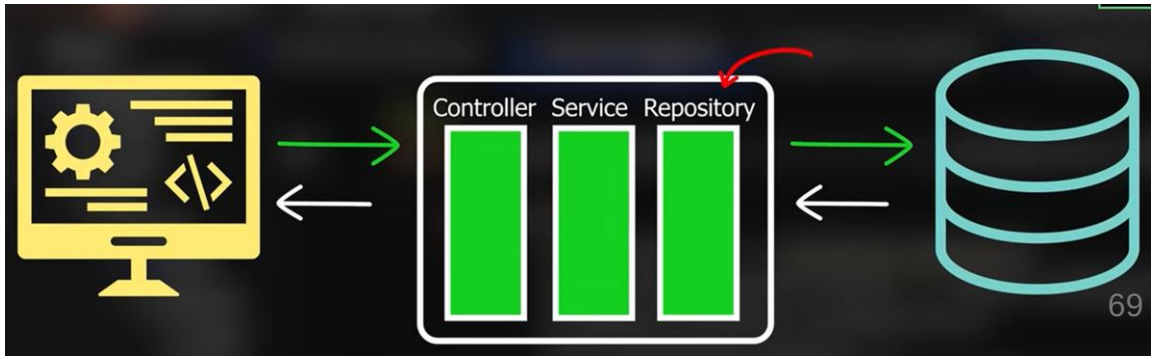
- This service is used by the SecurityConfig to configure the filter chain (see next slide).

```java
@Service
public class MyUserDetailsService implements UserDetailsService {

    @Autowired
    private UserRepository userRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        User user =  userRepository.findByUsername(username);
        if ( user == null ) {
            throw new UsernameNotFoundException("User not found");
        }
        return new UserPrincipal(user);
    }

}
```



Configure our security filter chain

```java
@Configuration // tell Spring this is a configuration class (Spring will scan this class for beans)
@EnableWebSecurity // replaces the default filter chain by the one configured here (Spring will scan this class for beans)
public class SecurityConfig {

    @Autowired
    private UserDetailsService userDetailsService;

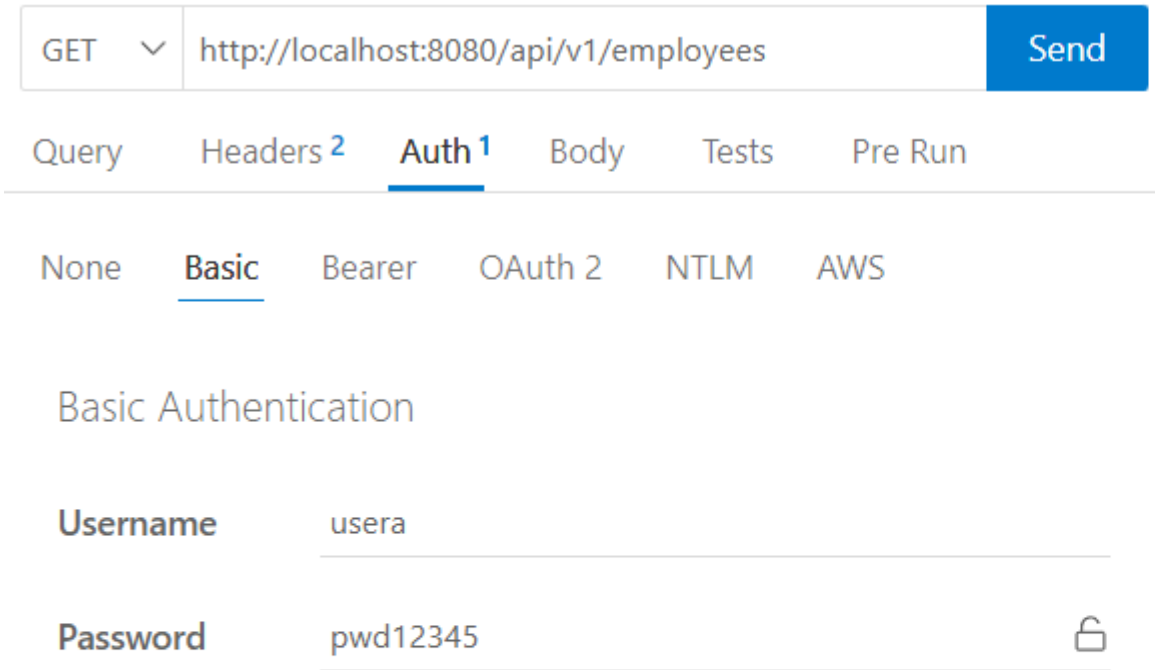
    @Bean
    public AuthenticationProvider authenticationProvider(PasswordEncoder passwordEncoder) {
        DaoAuthenticationProvider provider = new DaoAuthenticationProvider();
        provider.setPasswordEncoder(passwordEncoder);
        provider.setUserDetailsService(userDetailsService);
        return provider;
    }

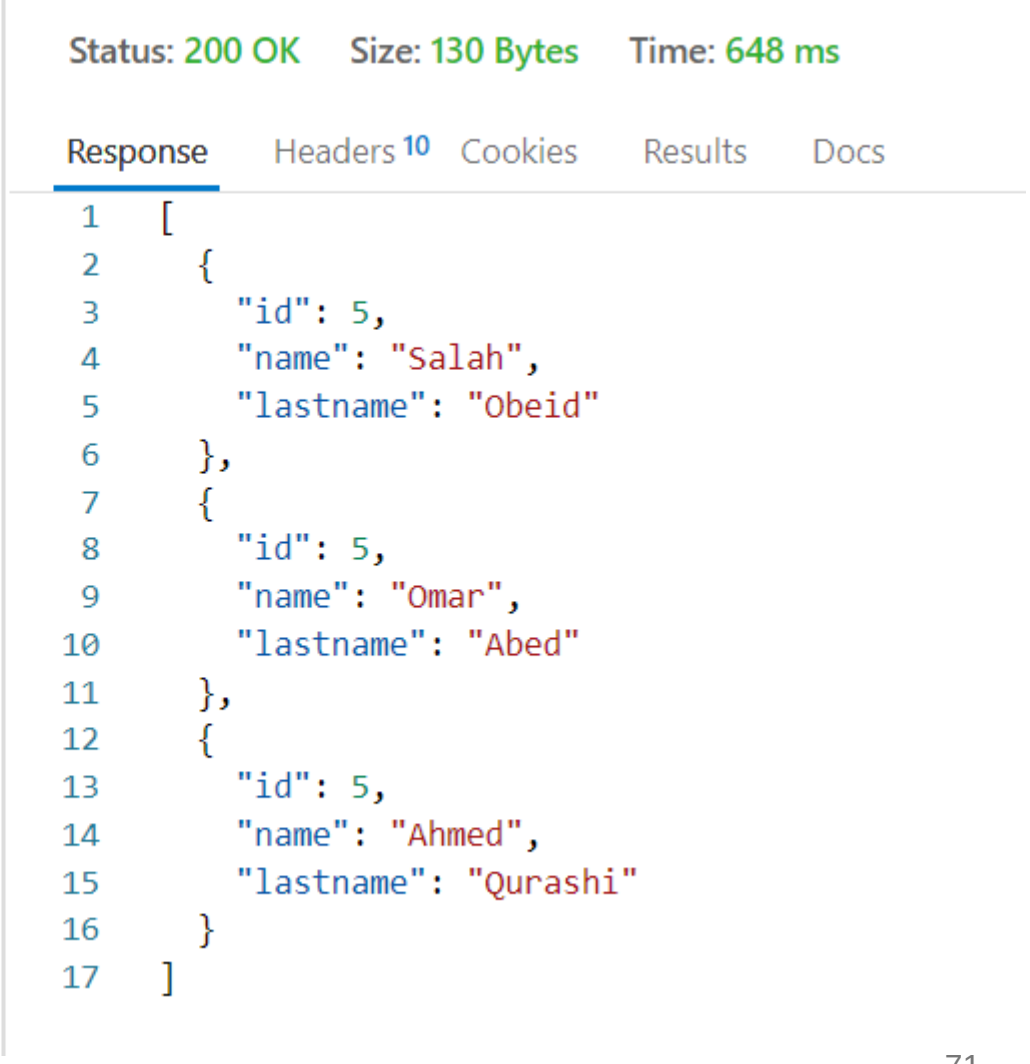
    @Bean // Spring will scan this method for beans
    public SecurityFilterChain securityFilterChain(HttpSecurity httpsec) throws Exception {
        httpsec.csrf(customizer -> customizer.disable());
        httpsec.authorizeHttpRequests(request -> request.anyRequest().authenticated());
        httpsec.httpBasic(Customizer.withDefaults());
        httpsec.sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS));
        return httpsec.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

Testing the API

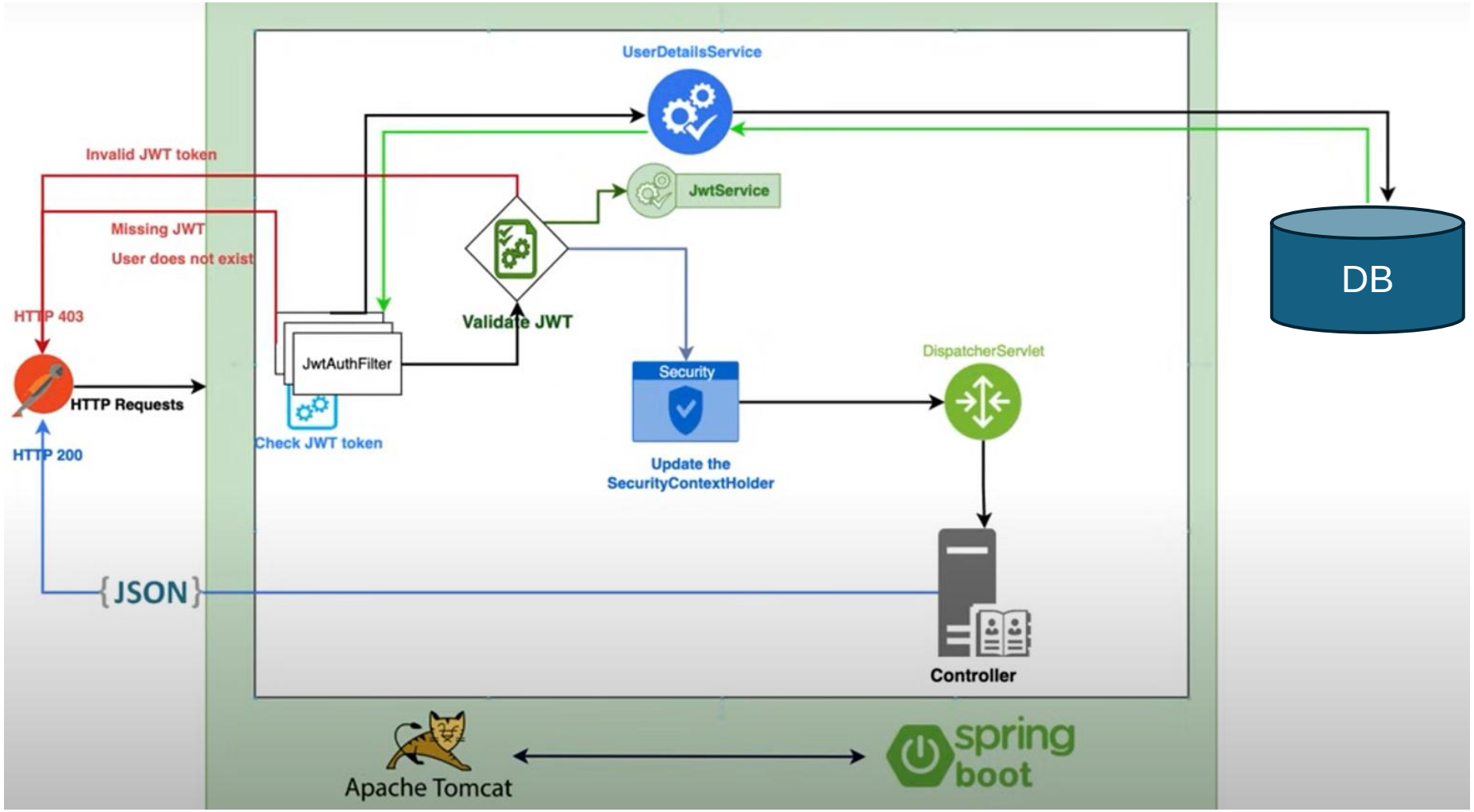




Authentication with JWT

Ref: https://www.youtube.com/watch?v=oeni_9g7too

JWT Authentication



JWT authentication in Spring Security works by using a **stateless, token-based mechanism** where a JSON Web Token (JWT) is issued upon successful login and then included in subsequent requests to access protected resources. Spring Security uses a custom **filter** to intercept and validate these tokens, setting the user's authentication context without requiring server-side session storage.

What is a JWT (JSON Web Token)

Encoded PASTE A TOKEN HERE

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey
JzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6Ikpva
G4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKx
wRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

JSON Web Token (JWT) is an open standard that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. JWTs can be signed using a secret (with the **HMAC** algorithm) or a public/private key pair using **RSA** or **ECDSA**.

Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

PAYLOAD: DATA

```
{
  "sub": "1234567890",
  "name": "John Doe",
  "iat": 1516239022
}
```

VERIFY SIGNATURE

```
HMACSHA256(
  base64UrlEncode(header) + "." +
  base64UrlEncode(payload),
  your-256-bit-secret
) ☐ secret base64 encoded
```

What is the JSON Web Token structure?

JSON Web Tokens consist of three parts separated by dots (`.`), which are: Header, Payload, Signature:

`xxxxx.yyyyy.zzzzz`

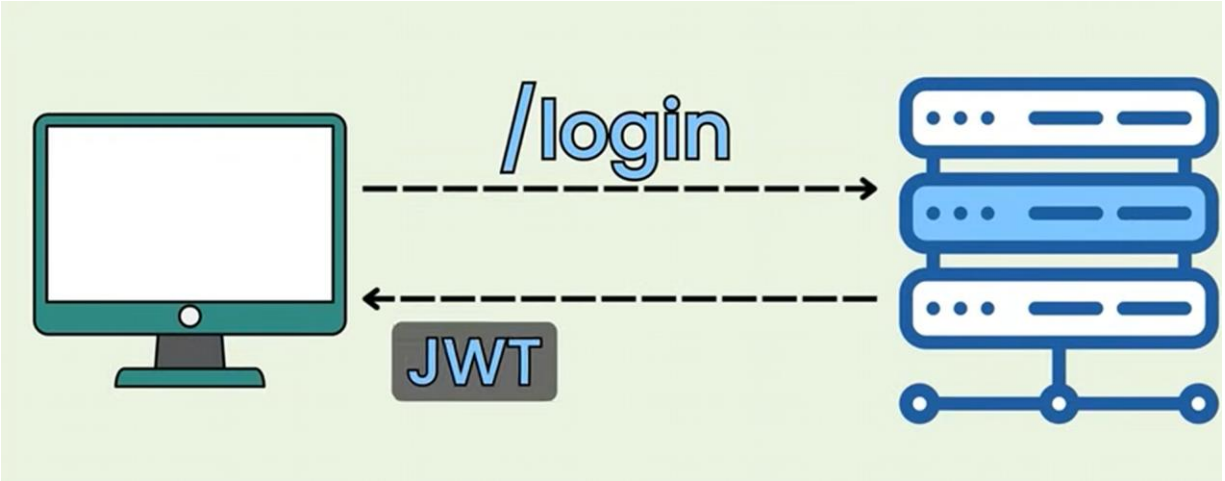
- **Header:** The header *typically* consists of two parts: the type of the token, which is JWT, and the signing algorithm being used, such as HMAC SHA256 or RSA.
  For example: `{ "alg": "HS256", "typ": "JWT" }`

- **Payload:** contains the **claims**. Claims are statements about an entity (typically, the user) and additional data.
  For example: `{ "sub": "1234567890", "name": "Salah Otaibi", "admin": true }`

- **Signature:** To create the signature part you have to take the encoded header, the encoded payload, a secret, the algorithm specified in the header, and sign that. The signature is used to verify the message wasn't changed along the way, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.

How do JSON Web Tokens work?

In JWT authentication, when the user successfully logs in using his credentials, a JWT Utility or Service class then generates a JWT containing public user details (claims), an expiration time, and a signature using a secret key. The generated JWT is returned to the client in the response body or as an HTTP-only cookie.

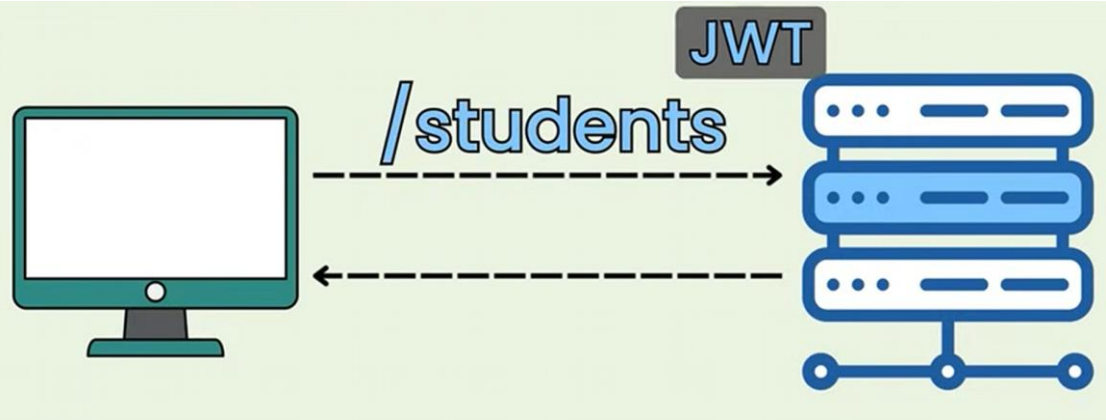


How do JSON Web Tokens work? (2)

Whenever the user wants to access a protected route or resource, the user agent (browser) sends the JWT, in the **Authorization** header using the **Bearer** schema or as a cookie. The content of the header should look like the following: `Authorization: Bearer <token>`

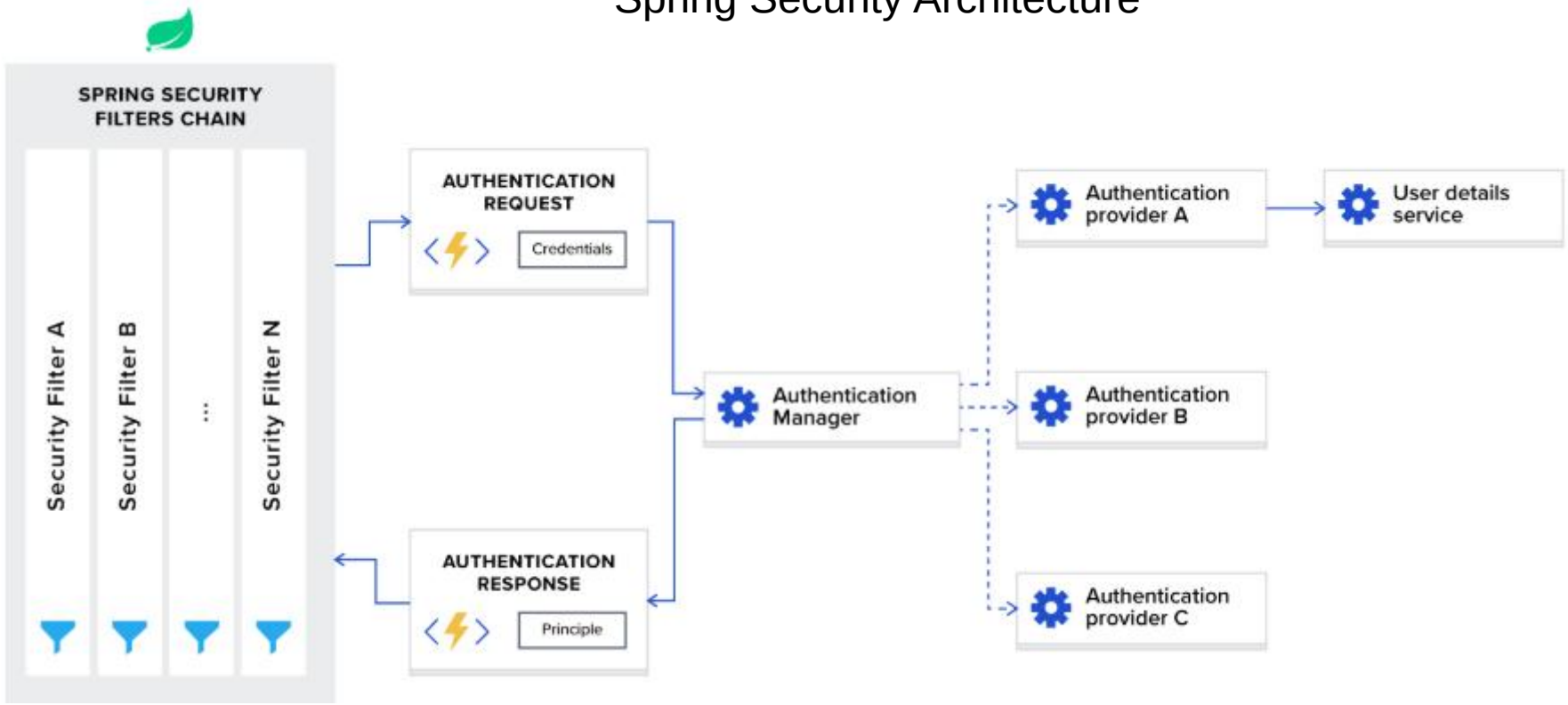
- This can be, in certain cases, a stateless authorization mechanism. The server's protected routes will check for a valid JWT in the `Authorization` header or cookie, and if it's present, Spring Security's filters then check the user's authorities against the access rules defined for the requested endpoint.
- If authorized, the request proceeds to the controller.
- If unauthorized, Spring Security returns an HTTP 401 (Unauthorized) or 403 (Forbidden) error.

This stateless approach allows for high scalability in microservice architectures because the server doesn't need to maintain session information for each user.



JWT example

Spring Security Architecture



`AuthenticationManager` is a coordinator where you can register multiple providers, and based on the request type, it will deliver an authentication request to the correct provider.

Configuration of the Spring App for JWT

We need to import the library API (Facade) and Implementation libraries

And Jackson (a Java library for processing JSON)

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.13.0</version>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>

<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.13.0</version>
  <scope>runtime</scope>
</dependency>
```

A JwtService for all token logic

```
@Service
public class JwtService {

  // Private key used to sign JWT tokens
  private String secret;

  public String generateToken(String username) {}

  public boolean validateToken(String token, UserDetails userDetails) {}
  public String extractUsername(String token) {}
  private <T> T extractClaim(String token, Function<Claims, T> claimResolver) {}

  private boolean isTokenExpired(String token) {}
  private Date extractExpiration(String token) {}


}
```

Generating the token

1- Create Claims Map: all the key/value pairs of the Token payload
2- Add subject claim
3- Add issue date claim
4- Add expiration date claim
5- Sign and build String

```
public String generateToken(String username) {
  Map<String, Object> claims = new HashMap<>();
  return Jwts.builder()
          .claims()
          .add(claims)
          .subject(username)
          .issuedAt(new Date(System.currentTimeMillis()))
          .expiration(new Date(System.currentTimeMillis() + 60 * 60 * 1000))
          .and()
          .signWith(getKey())
          .compact();
}
```

Secret key used to encode/decode tokens

```
// Private key used to sign JWT tokens
private String secret;


// Generate ephemeral key (new key for every server restart), key not stored permanently
// If server restarts all users need to re-login
public JwtService() {
  KeyGenerator keyGen;
  try {
      keyGen = KeyGenerator.getInstance("HmacSHA256");
      SecretKey sk = keyGen.generateKey();
      secret = Base64.getEncoder().encodeToString(sk.getEncoded());
  } catch (NoSuchAlgorithmException e) {
    e.printStackTrace();
  }
}
```

Validating the Token

```
public boolean validateToken(String token, UserDetails userDetails) {
  final String username = extractUsername(token);
  return (username.equals(userDetails.getUsername()) && !isTokenExpired(token));
}
public String extractUsername(String token) {
   return extractClaim(token, Claims::getSubject);
}

private <T> T extractClaim(String token, Function<Claims, T> claimResolver) {
  final Claims claims = extractAllClaims(token);
  return claimResolver.apply(claims);
}
private boolean isTokenExpired(String token) {
  return extractExpiration(token).before(new Date());
}

private Date extractExpiration(String token) {
  return extractClaim(token, Claims::getExpiration);
}
```

```
private Claims extractAllClaims(String token)
{
  return Jwts.parser()
       .verifyWith(getKey())
       .build()
       .parseSignedClaims(token)
       .getPayload();
}
```

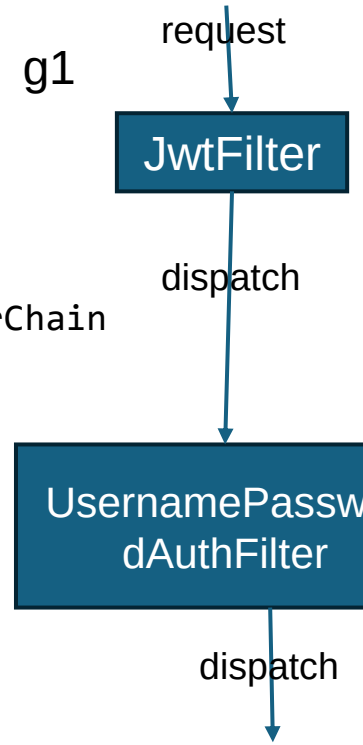
Inserting a JWT filter in the filter chain

- If a valid token exists in the incoming request, no need to check for basic auth (username/password), this is why the **JwtFilter** is added before the **UsernamePasswordAuthenticationFilter** (which is the filter created by httpBasic())

```
public class SecurityConfig {
  @Autowired
  private JwtFilter jwtFilter;


@Bean // Spring will scan this method for beans
public SecurityFilterChain securityFilterChain(HttpSecurity httpsec) throws Exception {
  return httpsec.csrf(customizer -> customizer.disable())
  .authorizeHttpRequests(request -> request
  .requestMatchers("register", "login").permitAll()
  .anyRequest().authenticated())
  .httpBasic(Customizer.withDefaults())
  .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
  .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
  .build();
}
```

The main component: The JwtFilter




```
@Component public class JwtFilter extends OncePerRequestFilter {
  @Autowired private JwtService jwtService;   @Autowired ApplicationContext context;

  protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain
    filterChain)  … {
      String authHeader = request.getHeader("Authorization");
      if(authHeader != null && authHeader.startsWith("Bearer ")) {
        token = authHeader.substring(7); // skip 7 characters (Bearer)
        username = jwtService.extractUsername(token);
      }
      // Check that username was found and that he is not already authenticated
      if(username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
        UserDetails userDetails = context.getBean(MyUserDetailsService.class).loadUserByUsername(username);
        if(jwtService.validateToken(token, userDetails)) {
                UsernamePasswordAuthenticationToken authToken = new UsernamePasswordAuthToken(userDetails, null,
                        userDetails.getAuthorities());
                authToken.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(authToken);
        }
      }
      filterChain.doFilter(request, response);   // Dispatch to the next filter
  }
}
```

UserController: register and login

```
@RestController
public class UserController {

  @Autowired
  private UserService userService;

  private BCryptPasswordEncoder encoder = new BCryptPasswordEncoder(12);

  @PostMapping("/register")
  public Users register(@RequestBody Users user) {
       user.setPassword(encoder.encode(user.getPassword()));
       userService.register(user);
       user.setPassword("");
       return user;
  }

  @PostMapping("/login")
  public String login(@RequestBody Users user) {
       return userService.verify(user);
  }
}
```

UserService: verifies login requests

```
@Service
public class UserService {

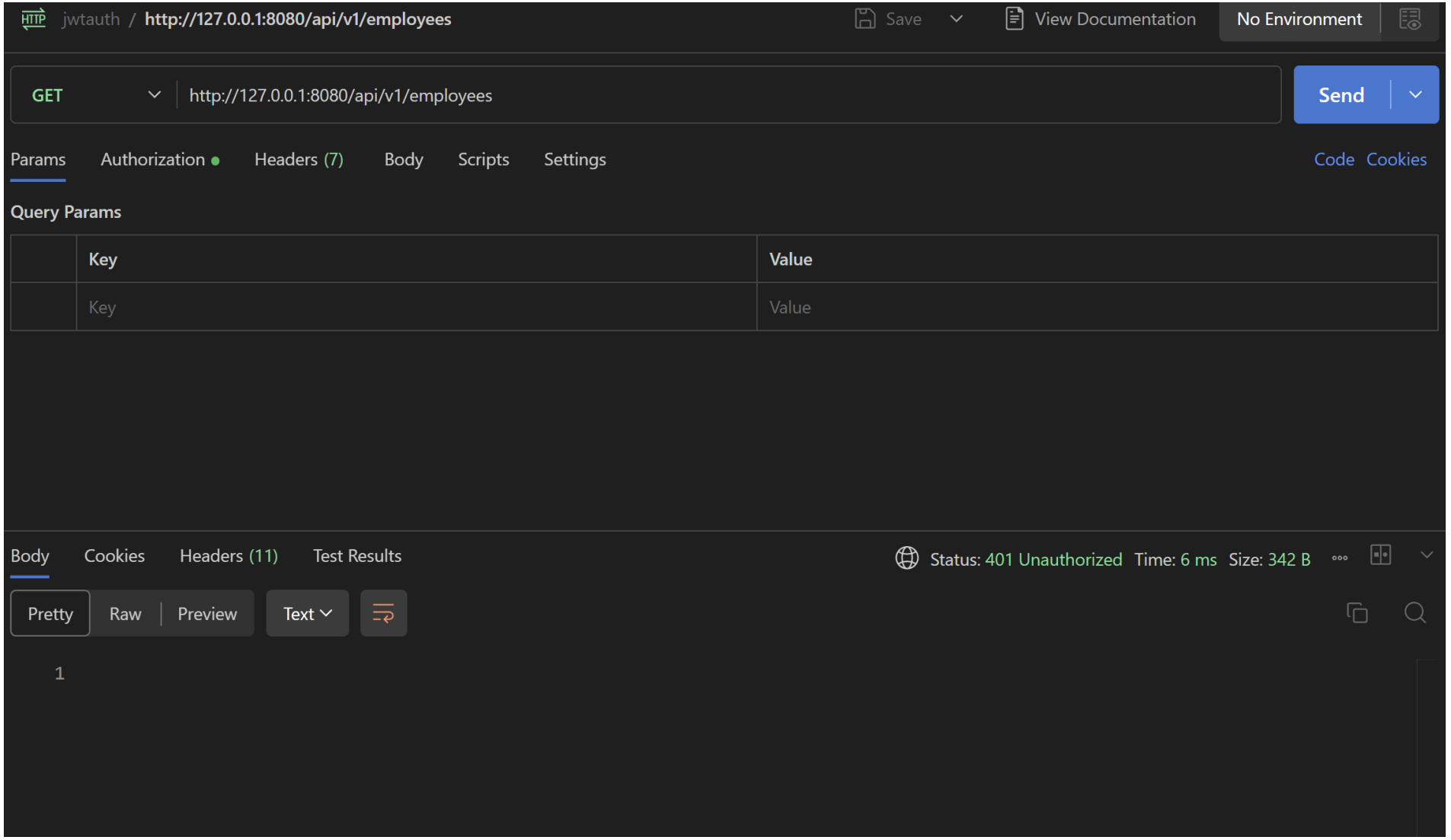
  @Autowired UserRepository userRepository;
  @Autowired AuthenticationManager authManager;
  @Autowired JwtService jwtService;

  public Users register(Users user) {
    return userRepository.save(user);
  }

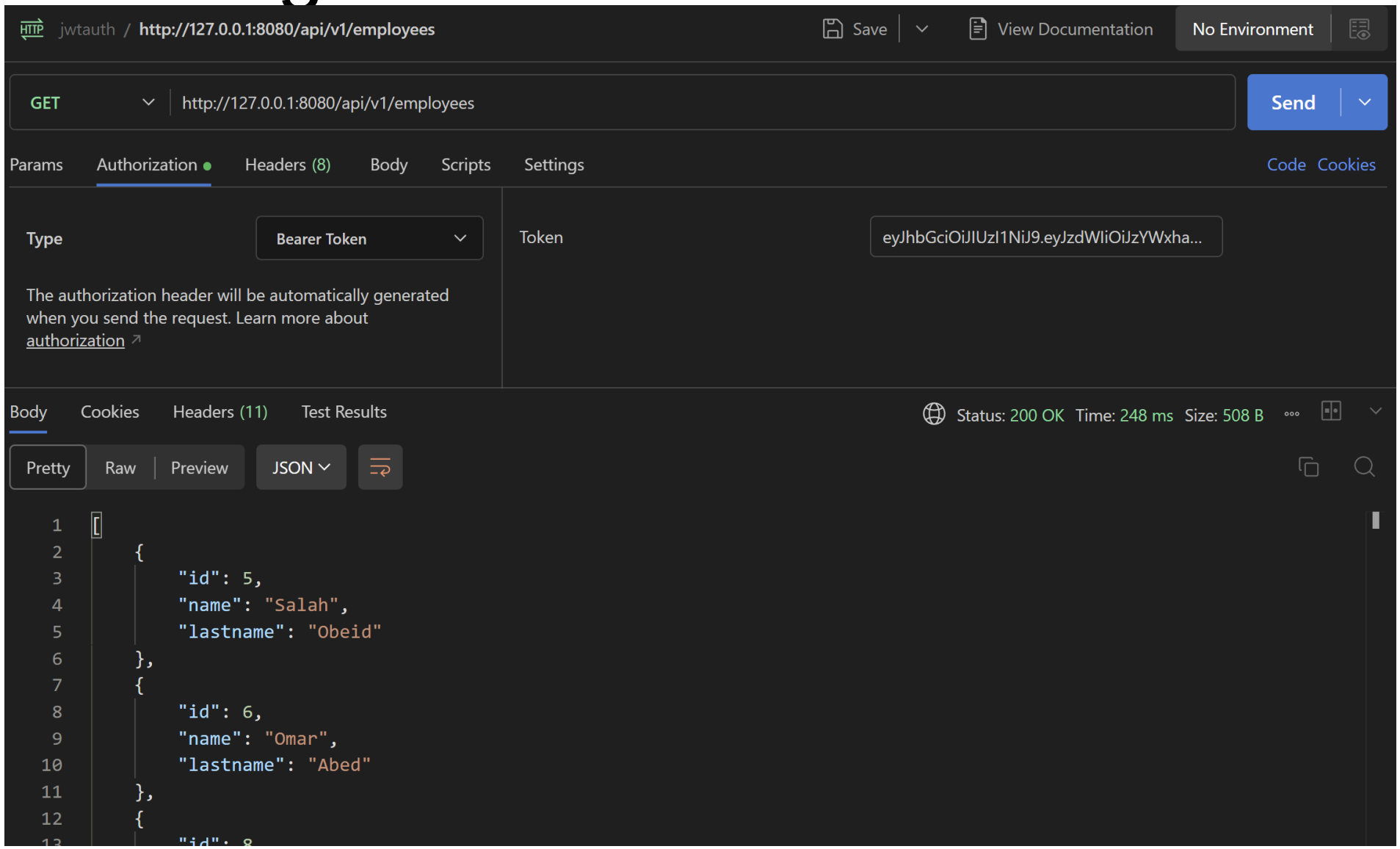

  public String verify(Users user) {
     Authentication auth = authManager.authenticate(new
         UsernamePasswordAuthToken(user.getUsername(), user.getPassword()));
     if(auth.isAuthenticated())
        return jwtService.generateToken(user.getUsername());

     return "Failed";
  }
}
```

Accessing API without token or basic auth



Accessing API with Bearer token



The client using the Token (localStorage)

1- At login, save the token immediately:

```
localStorage.setItem('jwt_token', received_token);
```

2- When sending any request to a protected resource:

```
const token = localStorage.getItem('jwt_token');

// The token received from /login fetch('http://localhost:8080/api/protected-resource', {
    method: 'GET',
    headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${token}`
    }
}) .then(response => response.json()) .then(data => console.log(data));
```

The Risk (XSS): Vulnerability: `localStorage` is accessible by *any* JavaScript running on your page. For example, if you use a third-party npm package that is compromised or if you accidentally render user input without escaping it, an attacker can read your local storage, steal the token, and impersonate the user.

The client using the Token (HttpOnly cookie)

- Instead of returning the string in the response body, set a Set-Cookie header. The browser then automatically handles the storage.
- **Server:** Sets the cookie as HttpOnly (JavaScript cannot read it) and Secure (HTTPS only).
- **Client:** You do **not** need to manually add the Authorization header. The browser automatically sends the cookie with every request to your domain.
- **Pros:** Immune to XSS. Even if malicious scripts run on your page, they cannot read document.cookie
- **Cons:** You are now vulnerable to **CSRF** (Cross-Site Request Forgery) and need to implement CSRF protection: the browser will automatically send that cookie with *every* request to your API, including requests initiated by a malicious site

```
@PostMapping("/login")
public ResponseEntity<String> login(@RequestBody Users user,
HttpServletResponse response) {
    String token = userService.verify(user);

    // Create the Cookie
    ResponseCookie jwtCookie = ResponseCookie.from("jwt", token)
            .httpOnly(true)       // JS cannot read this (Protects against
XSS)
            .secure(true)         // HTTPS only
            .path("/")
            .maxAge(24 * 60 * 60) // 1 day
            .sameSite("Strict")   // CRITICAL for CSRF protection
            .build();

    // Add to response header
    return ResponseEntity.ok()
            .header(HttpHeaders.SET_COOKIE, jwtCookie.toString())
            .body("Login successful");
}
```

JWT best practices

- JWT tokens cannot be altered if key is kept private
- JWT is not a “secret” -> never put secret information inside a token
- JWT is **not encrypted**, it is only **signed**
- Keep validity short (15 minutes, depends of app)
- Many front-end apps store JWT tokens in local storage, tokens can be stolen by XSS attacks:
  - Use HttpOnly cookies
  - Add a content security policy (CSP)
  - Sanitize user input
  - Use sub-resource integrity: Sometimes the malicious script doesn't come from user input, but from a library (like jQuery or Bootstrap) you are loading from a CDN: You generate a cryptographic hash of the *original* valid file and add it to your HTML <script> tag.

```
<script
  src="https://cdn.example.com/library.js"
  integrity="sha384-Li9vy3DqF8..."
  crossorigin="anonymous">
</script>
```

end